

Toán tối thiểu dùng lại suốt đời (The Minimal Mathematics You Reuse Forever)

PHỤ THUỘC không — đây là chương mở đầu của cây. Chương này chỉ nên đọc *theo nhu cầu*: quay lại đúng mục cần khi một chương ở Phần B, C hay D đòi hỏi, chứ không đọc một mạch từ đầu tới cuối.

TÓM TẮT

Chương này giữ lại đúng phần toán quay lại nhiều lần về sau, chia theo *vai trò* chứ không theo tên môn học. Mục 1 lo không gian và phổ. Mục 2 dựng bộ máy đưa mô hình từ khởi tạo tới nghiệm. Mục 3 nói nguồn gốc của hàm mất mát và độ tin cậy của một con số đo được. Mục 4 nói cấu trúc có sẵn trong dữ liệu ảnh. Đọc xong bạn chẩn đoán được nguyên nhân của một đường cong huấn luyện xấu, suy được loss từ giả định nhiễu thay vì tra bảng, và bắt được các lỗi im lặng ở khâu tiền xử lý. Nếu chỉ nhớ một điều: đây là chương [T1], nên dừng học cho đủ — hãy quay lại đúng mục mà một chương sau đang cần, vì kiến thức học không có nhu cầu đi kèm sẽ bay hết trong hai tuần.

KIẾN THỨC CẦN CÓ TRƯỚC

- **deep learning** — cách tiếp cận trong đó ta xếp chồng nhiều tầng biến đổi có tham số, rồi để dữ liệu tự quyết định các tham số đó thông qua tối ưu hoá, thay vì tự tay thiết kế đặc trưng.
- **mô hình, tham số, huấn luyện** — mô hình là một hàm có tham số nhận đầu vào và trả về dự đoán; tham số là các con số bên trong nó; huấn luyện là quá trình tìm bộ tham số làm dự đoán sát nhãn nhất.
- **hàm mất mát (loss function)** — một con số đo mức sai của dự đoán so với nhãn — đại lượng mà quá trình huấn luyện thực sự cực tiểu.
- **tensor** — mảng số nhiều chiều; một ảnh màu là tensor ba chiều (cao $\times$ rộng $\times$ kênh), một batch ảnh là tensor bốn chiều.
- **nhãn tầng bán rã [T1]–[T4]** — quy ước của cây này về tuổi thọ của tri thức: [T1] là toán và vật lý gần như không đổi; [T2] là nguyên lý thiết kế hệ thống; [T3] là thuật toán và mô hình; [T4] là thư viện và công cụ, cũ nhanh nhất.
- **SLAM và hiệu chỉnh chùm tia (bundle adjustment)** — SLAM là bài toán vừa dựng bản đồ vừa định vị trong chính bản đồ đó; hiệu chỉnh chùm tia là bước tinh chỉnh đồng thời mọi tư thế camera và điểm 3D bằng bình phương tối thiểu. Chương này dùng chúng làm điểm neo quen thuộc, không đòi hỏi bạn phải cài được chúng.

CÂU HỎI MỞ ĐẦU

- Q1. Nếu quy tắc chuỗi là thứ ai cũng học từ năm nhất, vì sao tính *gradient* cho một mạng 10^9 tham số lại từng là chuyện bất khả thi?
- Q2. Vì sao đổi đơn vị từ mét sang milimét trong một bài hiệu chỉnh chùm tia làm số điều kiện đổi ba bậc, trong khi bài toán không hề dễ hay khó đi?
- Q3. Hai mô hình chênh nhau một điểm phần trăm trên tập test 500 mẫu, mà khoảng tin cậy của mỗi mô hình rộng hơn năm điểm — vậy vì sao kết luận “A tốt hơn B” vẫn có thể đúng?
- Q4. Vì sao một mạng dự đoán phép quay bằng bốn số lại khó huấn luyện hơn bằng sáu số, dù bốn số đã thừa để mô tả ba bậc tự do?

Mọi lộ trình học deep learning đều bắt đầu bằng một chương toán, và phần lớn người học đều bỏ dở nó. Lý do không phải toán khó mà vì chương đó thường được viết như một giáo trình rút gọn: đầy đủ, tuyến tính, và không nói cho người đọc biết thứ nào sẽ quay lại vào lúc nào. Kết quả là người ta học đại số tuyến tính suốt sáu tuần rồi vẫn không biết vì sao huấn luyện của mình không hội tụ. Chương này cố ý làm ngược lại: chỉ giữ những gì *thật sự* quay lại nhiều lần, và mỗi mục phải trả lời được câu “thứ này xuất hiện lại ở đâu”.

Cách chia bốn mục theo vai trò, và giữa chúng có phụ thuộc thật chứ không phải thứ tự tùy tiện:

- **Mục 1 — đại số tuyến tính:** bộ từ vựng về không gian và phổ. Trả lời “bài toán này suy biến ở hướng nào, tôi mất bao nhiêu chữ số”.
- **Mục 2 — vi phân và tối ưu hoá:** bộ máy đưa mô hình từ khởi tạo tới nghiệm. *Phụ thuộc mục 1*, vì “hình dạng landscape” chính là phổ trị riêng của Hessian nói bằng lời khác.
- **Mục 3 — xác suất và lý thuyết thông tin:** trả lời hai câu mà hai mục trước không đụng tới — hàm mất mát từ đâu ra, và con số đo được đáng tin đến đâu. *Tiếp nhận câu hỏi mà mục 2 để ngỏ*.
- **Mục 4 — hình học và tín hiệu:** khác hẳn ba mục kia ở chỗ nó không nói về *cách học* mà về *cấu trúc có sẵn trong dữ liệu ảnh*. Đọc được đọc lập, và là cầu nối trực tiếp sang chương A/02.

Thứ tự đọc đề nghị là $1 \rightarrow 2 \rightarrow 3$, còn mục 4 đọc được bất cứ lúc nào. Với người đã làm SLAM bằng C++, phần lớn mục 1 và nửa đầu mục 4 là ôn tập; bốn chỗ đáng dừng lại vì kinh nghiệm ước lượng trạng thái *không* chuyển sang miễn phí là: số điều kiện đọc theo nghĩa tốc độ huấn luyện, điều kiện dừng theo validation, toàn bộ mục 3, và phần lấy mẫu của mục 4.

Bản đồ chương — Toán tối thiểu dùng lại suốt đời

TÓM TẮT

Bản đồ này nói chương 1 gồm bốn mục nào, và vì sao chia theo vai trò chứ không theo tên môn học. Nó cũng nói mục nào phụ thuộc mục nào, và phần nào là cốt lõi so với phần nào chỉ cần biết là có. Đọc xong bạn quyết định được trong một phút nên đọc mục nào trước cho nhu cầu hiện tại thay vì đọc tuần tự cả chương. Nếu chỉ nhớ một điều: đây là chương [T1], nên nguyên tắc là học theo nhu cầu. Chuỗi phụ thuộc bắt buộc chỉ là $1 \rightarrow 2 \rightarrow 3$. Mục 4 vào được bất cứ lúc nào, và nên đọc ngay trước chương A/02.

A. Chương này có gì

- **01-dai-so-tuyen-tinh** — không gian con, phép chiếu, phân rã trị riêng và SVD, số điều kiện, xấp xỉ hạng thấp.
- **02-vi-phan-va-toi-uu** — quy tắc chuỗi dạng ma trận, Jacobian, Hessian, hạ gradient, lỗi so với không lồi, điều kiện dừng, hình dạng landscape.
- **03-xac-suat-va-thong-tin** — sai số lấy mẫu và khoảng tin cậy, MLE và MAP như nguồn gốc chung của mọi hàm mất mát, entropy-cross-entropy-KL và sự bất đối xứng của KL.
- **04-hinh-hoc-va-tin-hieu** — toạ độ đồng nhất, $SO(3)$ và $SE(3)$ ở mức ba thao tác dùng được, convolution, Fourier, lấy mẫu và aliasing.

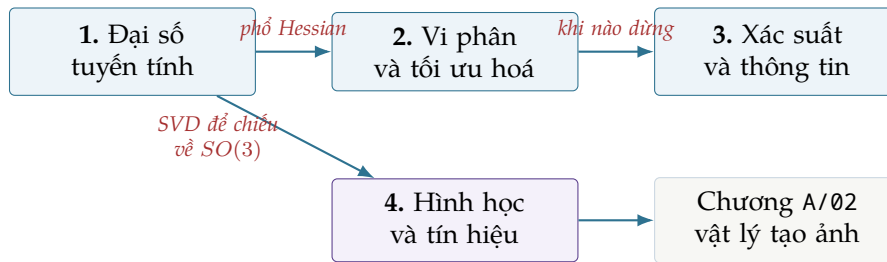
B. Vì sao chia như vậy

Cách chia thông thường theo tên môn học — “đại số tuyến tính, giải tích, xác suất” — không nói thứ nào quay lại vào lúc nào. Nó vì thế khuyến khích học tuyến tính cho đủ, rồi quên sạch. Bốn

mục ở đây được đặt theo *câu hỏi chúng trả lời*: mục 1 trả lời “bài toán này suy biến ở hướng nào và tôi mất bao nhiêu chữ số”; mục 2 trả lời “làm sao đi từ khởi tạo tới nghiệm, và cái gì làm nó chậm”; mục 3 trả lời “hàm mất mát từ đâu ra, và con số tôi vừa đo đáng tin đến đâu”; mục 4 khác hẳn ba mục kia vì nó không nói về cách học mà về *cấu trúc có sẵn trong dữ liệu ảnh trước khi mô hình nhìn thấy*.

C. Phụ thuộc và thứ tự đọc

Hình 1.1 vẽ quan hệ phụ thuộc thật giữa bốn mục.



Hình 1.1. Phụ thuộc giữa bốn mục của chương 1. Chuỗi ngang là chuỗi bắt buộc; mục 4 (tô khác màu) chỉ mượn một kết quả nhỏ của mục 1 nên vào được bất cứ lúc nào, và nó là cửa dẫn sang chương A/02.

D. Cốt lõi hay tham khảo

Toàn chương thuộc tầng [T1], tức học theo nhu cầu và không đầu tư sâu. Trong đó:

- **Cốt lõi — phải nắm chắc, sẽ quay lại nhiều lần:** số điều kiện và xấp xỉ hạng thấp (mục 1); quy tắc khớp shape, chi phí bộ nhớ của lan truyền ngược, quan hệ độ cong–learning rate (mục 2); *toàn bộ* mục 3; phần lấy mẫu và aliasing (mục 4).
- **Tham khảo — biết là có, tra khi cần:** chi tiết các phân rã ma trận, phân tích lỗi hình thức, phần Lie algebra vượt quá ba thao tác exp, log và cập nhật trên không gian tiếp tuyến.

E. Riêng với người đọc đã làm SLAM

Bốn chỗ mà kinh nghiệm ước lượng trạng thái *không* chuyển sang miễn phí, nên dừng lại kỹ: (i) số điều kiện đọc theo nghĩa tốc độ huấn luyện chứ không chỉ theo nghĩa ổn định số học; (ii) điều kiện dừng theo validation thay cho chuẩn gradient; (iii) toàn bộ mục 3, vì ước lượng trạng thái hiếm khi phải trả lời câu “hiệu số này có ý nghĩa thống kê không”; (iv) aliasing trong kiến trúc mạng, vốn không có đối ứng trong pipeline hình học cổ điển. Ngược lại, tọa độ đồng nhất, $SE(3)$ và bình phương tối thiểu thì đọc lướt được.

I 1.1 Đại số tuyến tính ứng dụng (Applied Linear Algebra)

PHỤ THUỘC không — đây là mục mở đầu của cây. Nếu bạn đã cài hiệu chỉnh chùm tia (*bundle adjustment*) thì phần không gian con là ôn lại; chỗ đáng đọc kỹ là *số điều kiện* và *hạng hiệu dụng*, hai chỗ mà ngôn ngữ SLAM và ngôn ngữ deep learning gọi cùng một hiện tượng bằng hai cái tên khác nhau.

TÓM TẮT

Mục này rút đại số tuyến tính xuống đúng bốn thứ quay lại suốt cây: không gian con và phép chiếu, hai phân rã eigen và SVD, số điều kiện, xấp xỉ hạng thấp. Đọc xong bạn phân biệt được “huấn luyện

chậm vì bài toán điều hoà tệ” với “chậm vì learning rate sai”. Bạn đọc được phổ trị kỳ dị của một tầng đặc trưng. Và bạn biết vì sao một solver nghiêm túc không bao giờ lập $A^T A$. Toàn bộ được neo vào thứ bạn đã làm được: hiệu chỉnh chùm tia, ellipsoid hiệp phương sai, gauge freedom. Nếu chỉ nhớ một điều: bình phương tối thiểu là một phép chiếu vuông góc, còn số điều kiện là hệ số khuếch đại sai số của *bài toán* chứ không phải của thuật toán. Hệ quả: đối optimizer không cứu được một bài toán bị điều hoà tệ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **Quy ước ký hiệu** — vector luôn là vector *cột*; A^T là ma trận chuyển vị; ma trận $m \times n$ có m hàng và n cột; $\|\cdot\|$ không ghi chỉ số nghĩa là chuẩn Euclid (chuẩn 2).
- **mô hình (model)** — một hàm có tham số, nhận đầu vào và trả về dự đoán. “Huấn luyện” nghĩa là đi tìm bộ tham số làm dự đoán sát nhãn nhất.
- **trọng số (weight)** — chính là các tham số của mô hình. Trong một tầng tuyến tính, chúng được xếp thành ma trận W và phép tính là $Wx + b$.
- **gradient** — vector gồm các đạo hàm riêng của một hàm theo từng tham số; nó chỉ hướng làm hàm tăng nhanh nhất, nên thuật toán tối ưu đi *ngược* hướng đó.
- **ma trận Hessian** — ma trận đạo hàm bậc hai; nó đo *độ cong* của hàm — hướng nào dốc gắt, hướng nào phẳng.
- **trị riêng và vector riêng** — vector riêng là hướng mà ma trận chỉ kéo dãn chứ không xoay; trị riêng là hệ số kéo dãn theo hướng đó. Tập hợp các trị riêng gọi là *phổ* của ma trận.
- **hạng (rank)** — số chiều thật sự mà đầu ra của ma trận trải ra. Hạng thấp nghĩa là ma trận nén dữ liệu về một không gian con nhỏ hơn.
- **hiệu chỉnh chùm tia (bundle adjustment)** — trong SLAM và dựng lại 3D, bài toán tinh chỉnh đồng thời mọi tư thế camera và mọi điểm 3D bằng cách cực tiểu tổng bình phương sai số tái chiếu. Nó là ví dụ chuẩn của bình phương tối thiểu phi tuyến quy mô lớn.

Đọc nhanh. Nếu bốn ý trên đã rõ, đọc thẳng tiểu mục D (số điều kiện) và E (giới hạn).

A. Đặt vấn đề

Một kỹ sư SLAM giải hệ tuyến tính mỗi ngày mà không gọi tên nó là đại số tuyến tính: mỗi vòng *Gauss–Newton* của hiệu chỉnh chùm tia là một bài bình phương tối thiểu. Mỗi ma trận **hiệp phương sai** (*covariance*) của keyframe là một dạng toàn phương đối xứng. Và mỗi lần khôi phục pose từ ma trận cơ bản là một lần gọi **phân rã giá trị kỳ dị** (*singular value decomposition, SVD*). Khi bước sang deep learning, những đối tượng ấy quay lại nguyên vẹn nhưng dưới tên khác: “không gian đặc trưng” thay cho không gian cột, **rank collapse** thay cho ma trận suy biến, “*ill-conditioned loss*” thay cho số điều kiện lớn. Vấn đề không phải thiếu kiến thức mà thiếu ánh xạ giữa hai bộ từ vựng.

Học xong mục này thì làm được gì mà trước đó không làm được? Ba việc rất cụ thể: (i) nhìn một đường cong huấn luyện chậm và phân biệt được “bài toán bị điều hoà tệ” với “learning rate sai” với “mô hình quá nhỏ”; (ii) đọc một biểu đồ phổ trị kỳ dị của tầng đặc trưng và nói được mô hình đang thu hẹp biểu diễn hay không; (iii) biết vì sao một thư viện giải bài bình phương tối thiểu nghiêm túc không bao giờ lập ma trận $A^T A$.

Mục này không dạy lại đại số tuyến tính. Nó chọn ra bốn ý sẽ quay lại suốt Phần B, C, D và cho mỗi ý một chỗ neo trong thứ bạn đã làm được bằng C++:

- **Bốn ý được giữ lại, và câu hỏi mà mỗi ý trả lời:**
 - **không gian con** (*subspace*) và **phép chiếu** (*projection*) — “nghiệm bình phương tối thiểu là cái gì, chứ không phải tính bằng công thức nào”.

- **phân rã trị riêng** (*eigendecomposition*) và SVD — “ma trận này kéo dãn không gian theo những trục nào, mạnh yếu ra sao”.
- **số điều kiện** (*condition number*) — “tôi sắp mất bao nhiêu chữ số, và hướng nào của bài toán gần như không bị ràng buộc”.
- **xấp xỉ hạng thấp** (*low-rank approximation*) — “giữ lại bao nhiêu là đủ, và bỏ đi cái gì”.

Nguyên tắc của cả Phần A áp dụng nguyên: chỉ tới mức dùng được và giải thích được, không chứng minh định lý. Ai muốn chứng minh thì đọc [1]; bản trình bày định hướng học máy ở [2].

TRỰC GIÁC

Bạn đã nhìn thấy SVD mỗi lần vẽ ellipsoid bất định của một keyframe mà không biết. Ellipsoid đó là hình ảnh của quả cầu đơn vị sau khi bị ma trận hiệp phương sai kéo dãn: hướng các trục chính là các **vector riêng** (*eigenvector*), độ dài các trục là căn của các **trị riêng** (*eigenvalue*). Mọi ma trận đều làm đúng ba động tác ấy với quả cầu đơn vị — xoay, kéo dãn theo vài trục ưu tiên, rồi xoay lần nữa — và SVD chỉ là biên bản ghi lại ba động tác. Khi ellipsoid dẹt gần thành đĩa, bài toán suy biến theo một hướng, và đó chính là lúc số điều kiện lớn.

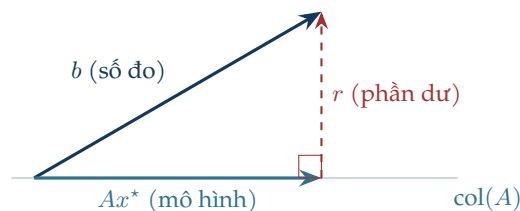
B. Không gian con và phép chiếu: bình phương tối thiểu không phải một công thức

Bình phương tối thiểu (*least squares*) thường được nhớ như công thức $x^* = (A^T A)^{-1} A^T b$. Nhớ như vậy thì không giải thích được vì sao nó hoạt động và hỏng khi nào. Cách nhớ dùng được lâu hơn là hình học: cột của A căng ra một không gian con của $\mathbb{R}^m$; vector đo b nói chung không nằm trong đó vì có nhiều; nghiệm bình phương tối thiểu là *hình chiếu vuông góc* của b xuống không gian cột, và phần dư $r = b - Ax^*$ vuông góc với mọi cột của A . Điều kiện vuông góc viết ra là

$$A^T r = 0 \iff A^T A x = A^T b.$$

Nói bằng lời: *phương trình chuẩn tắc không phải một mẹo đại số, nó chỉ là cách viết lại câu “phần dư phải vuông góc với mọi thứ mô hình giải thích được”*. Công thức là hệ quả, hình học là nguyên nhân.

Ví dụ nhỏ nhất tính tay được. Khớp một hằng số c vào ba số đo 1, 2, 6. Ở đây $A = (1, 1, 1)^T$, không gian cột là đường chéo. Chiều $b = (1, 2, 6)^T$ xuống đường ấy cho $c = (1 + 2 + 6)/3 = 3$, phần dư là $(-2, -1, 3)^T$, tổng bằng 0 — đúng là vuông góc với $(1, 1, 1)^T$. Trung bình cộng không phải một quy ước thống kê; nó là một phép chiếu. Hình 1.2 vẽ lại quan hệ đó ở dạng hai chiều.



Hình 1.2. Nghiệm bình phương tối thiểu là hình chiếu vuông góc của số đo b xuống không gian cột của A . Mọi biến thể — có trọng số, có ràng buộc, phi tuyến sau khi tuyến tính hoá — giữ nguyên bức tranh này và chỉ đổi cách đo góc vuông.

Bài toán phi tuyến trong SLAM không nằm ngoài bức tranh đó: Gauss–Newton tuyến tính hoá phần dư quanh điểm hiện tại, giải một bài chiếu, đi một bước, lặp lại. Ma trận $H = J^T J$ mà bạn quen gọi là “Hessian xấp xỉ” chính là $A^T A$ của bài chiếu ấy. Trong deep learning nó xuất hiện lại dưới tên **Gauss–Newton matrix**, và là bộ khung của mọi phương pháp bậc hai xấp xỉ. Đây là chỗ nối đầu tiên trong nhiều chỗ nối: cùng một ma trận, hai cộng đồng, hai cái tên.

I Mạch 1 — từ phép chiếu tới các phân rã

- 1 Vấn đề** giải $A^\top Ax = A^\top b$ trực tiếp mất chính xác một cách khó hiểu.

Giải pháp không lập $A^\top A$ mà phân rã A bằng **QR** hoặc SVD rồi giải trên dạng đã phân rã.

Hệ quả trị kỳ dị của $A^\top A$ là bình phương trị kỳ dị của A , nên số điều kiện bị bình phương và bạn mất khoảng một nửa số chữ số có nghĩa.
- 2 Vấn đề** nhưng hiệu chỉnh chùm tia vẫn lập $J^\top J$ mỗi vòng.

Giải pháp bài toán được điều hoà bằng **giảm chấn** (*damping*) kiểu Levenberg–Marquardt, và cấu trúc thưa làm **Cholesky** rẻ hơn QR nhiều bậc.

Giá phải trả bạn đánh đổi độ chính xác số học lấy tốc độ, và trả giá đúng lúc bài toán suy biến — ví dụ chuỗi keyframe gần thẳng hàng.
- 3 Vấn đề** cần biết “hướng nào của bài toán yếu”.

Giải pháp phân rã trị riêng của ma trận đối xứng H , hoặc SVD của J .

Hệ quả trị riêng nhỏ nhất chỉ ra hướng mà dữ liệu gần như không ràng buộc: trong SLAM đó là *gauge freedom*, trong deep learning đó là hướng phẳng của **bề mặt hàm mất mát** (*loss landscape*) mà **hạ gradient** (*gradient descent*) đi rất chậm.

C. Ba phân rã, chọn cái nào

Ba công cụ hay bị dùng lẫn. Bảng 1.1 đặt chúng cạnh nhau; cột cuối là cột đáng giá nhất.

Bảng 1.1. Ba phân rã ma trận dùng nhiều nhất, so trên cùng tiêu chí. $m \times n$ với $m \geq n$.

Phân rã	Cơ chế	Chi phí	Mạnh khi nào	Hỏng / không dùng được khi
Cholesky	$H = LL^\top$ trên ma trận đối xứng xác định dương	$\frac{1}{3}n^3$, rất rẻ khi thưa	Bài toán lớn, thưa, đủ điều hoà — lựa chọn mặc định trong BA	H suy biến hoặc gần suy biến; đã bình phương số điều kiện từ trước
QR	Trực giao hoá cột của A	$\approx 2mn^2$	Bình phương tối thiểu cần độ chính xác, A điều hoà kém	Ma trận rất lớn; không cho biết phổ trị kỳ dị
SVD	$A = U\Sigma V^\top$, mọi ma trận	$\approx O(mn^2)$, đắt nhất	Cần biết hạng, cần xấp xỉ hạng thấp, cần chiếu về $SO(3)$	Kích thước lớn (phải dùng phương pháp lặp hoặc phác thảo ngẫu nhiên)

Quan hệ giữa phân rã trị riêng và SVD gọn trong một câu: trị kỳ dị của A là căn bậc hai của trị riêng của $A^\top A$, và vector kỳ dị phải là vector riêng của $A^\top A$. Phân rã trị riêng chỉ đẹp khi ma trận đối xứng (hiệp phương sai, Hessian, ma trận Gram của đặc trưng); SVD tồn tại cho mọi ma trận nên nó là lựa chọn mặc định khi ma trận là một phép biến đổi giữa hai không gian khác nhau, chẳng hạn ma trận trọng số của một tầng tuyến tính.

Hệ quả thực dụng dùng lại nhiều nhất là **định lý xấp xỉ hạng thấp**: cắt bỏ các trị kỳ dị nhỏ cho ra ma trận hạng k gần A nhất. Ba chỗ trong cây này sống trên nó:

- **Nén và phân tích biểu diễn — phân tích thành phần chính** (*principal component analysis, PCA*) chỉ là chiếu dữ liệu đã trừ trung bình lên k vector kỳ dị phải đầu tiên.

- **Ép về cấu trúc hợp lệ** — ước lượng ma trận cơ bản rồi ép trị kỳ dị thứ ba về không; chiếu một ma trận nhiều về $SO(3)$ bằng cách đặt cả ba trị kỳ dị bằng một. Đây là xấp xỉ hạng thấp có ràng buộc, và nó quay lại ở A/01/04 khi mạng phải xuất ra một phép quay.
- **Tinh chỉnh rẻ** — *LoRA* [3] giả thiết phần cập nhật trọng số khi tinh chỉnh có hạng thấp, nên chỉ học hai ma trận gầy thay vì cả ma trận đầy.

↔ **trường hợp riêng của C** — LoRA áp dụng định lý xấp xỉ hạng thấp, nhưng cho ma trận *cập nhật trọng số* thay vì cho ma trận dữ liệu

D. Số điều kiện: đại lượng dự báo rắc rối

Số điều kiện $\kappa(A) = \sigma_{\max}/\sigma_{\min}$ trả lời đúng một câu hỏi: nếu đầu vào sai lệch tương đối ε , nghiệm có thể sai lệch tương đối tới $\kappa\varepsilon$. Nói bằng lời: κ là *hệ số khuếch đại sai số của bài toán, không phải của thuật toán*.

Ví dụ tính tay. Với $A = \text{diag}(1; 0,01)$ thì $\kappa = 100$; một nhiễu 1% đặt đúng vào thành phần thứ hai của b làm thành phần thứ hai của nghiệm sai 100%. Ma trận không hề suy biến, định thức khác không, mọi phép tính vẫn chạy — và kết quả vẫn vô nghĩa. Đó là lý do định thức gần như vô dụng làm chỉ báo, còn κ thì không.

Trong deep learning cùng đại lượng ấy chi phối tốc độ hội tụ, và gần như mọi “mẹo huấn luyện” quen thuộc là một cách hạ κ gián tiếp:

- **Chuẩn hoá đầu vào và chuẩn hoá theo batch (*batch normalization*)** [4] — kéo các hướng về cùng thang đo, tức trực tiếp giảm tỉ số trị riêng lớn nhất trên nhỏ nhất.
- *Adam* [5] — chia mỗi toạ độ cho một ước lượng độ lớn riêng, tương đương đổi thang đo từng chiều, nên nó “thấy” một bài toán điều hoà tốt hơn bài toán thật.
- **Khởi tạo cẩn thận** — giữ phổ trị kỳ dị của từng tầng không quá lệch ngay từ đầu, để κ không tích lũy qua độ sâu.

↔ **hệ quả của D** — chuẩn hoá, khởi tạo và optimizer thích nghi ở chương huấn luyện đều là cách hạ số điều kiện một cách gián tiếp

Một câu để mang theo: *phần lớn cái gọi là mẹo huấn luyện là kỹ thuật điều hoà số học đội lốt*. Mạch này quay lại đầy đủ ở D khi bàn optimizer.

E. Giới hạn và trường hợp hỏng

Bốn công cụ trên có giới hạn rõ, và biết giới hạn quan trọng hơn biết công thức.

- **Số điều kiện phụ thuộc đơn vị đo.** Trong một bài BA trộn tịnh tiến tính bằng mét với góc quay tính bằng radian, đổi mét sang milimét làm κ đổi ba bậc mà bài toán không dễ hay khó đi. So sánh κ giữa hai mô hình tham số hoá khác nhau là so sánh vô nghĩa.
- **Ma trận suy biến không phải lúc nào cũng là bệnh.** Hessian của BA không ràng buộc gauge suy biến đúng bảy chiều — đó là cấu trúc thật của bài toán. Hessian của mạng nơ-ron cũng suy biến vì nhiều đối xứng nội tại: hoán vị neuron trong một tầng, co giãn bù trừ giữa hai tầng liên tiếp qua *ReLU*. Cách xử lý đúng là cố định gauge hoặc chấp nhận, không phải thêm nhiễu cho đủ hạng.
- **PCA giả định cấu trúc tuyến tính và đo bằng phương sai.** Đặc trưng của mạng sâu nằm trên một đa tạp cong. Vì thế “95% phương sai trong k chiều” không có nghĩa thông tin hữu ích nằm ở đó. Hướng phương sai lớn thường là độ sáng toàn cục hoặc phong cách ảnh; hướng quyết định nhân có thể nằm ở phương sai nhỏ.

↔ **tiền đề E** — *linear probe* là bài bình phương tối thiểu trên không gian đặc trưng, nên chất lượng của nó bị chi phối bởi chính số điều kiện ở đây

Vì thể trực quan hoá đặc trưng hiện đại dùng *t-SNE* [6] hay *UMAP* [7]. Nhưng hai thứ đó đánh đổi tính trung thực về khoảng cách toàn cục — một trade-off mỗ ở E.

- **Chi phí.** SVD đầy đủ của ma trận $50\,000 \times 2048$ là chuyện của phương pháp lập hoặc phác thảo ngẫu nhiên, không phải một lời gọi thư viện vô tư.

CẠM BÃY

Đừng đọc “hạng thấp” như “ít thông tin”. Ma trận đặc trưng của một mô hình tự giám sát bị *rank collapse* là bệnh; ma trận cập nhật LoRA có hạng thấp là giả thiết thiết kế có chủ đích. Cùng một con số hạng, hai ý nghĩa trái ngược, và cái phân biệt chúng là *ma trận đó đang biểu diễn dữ liệu hay biểu diễn một phép hiệu chỉnh*. Trong thực hành nên đo **hạng hiệu dụng**, tức entropy của phổ trị kỳ dị đã chuẩn hoá. Đừng đếm số trị kỳ dị khác không: với số dấu phẩy động thì hầu như không trị kỳ dị nào đúng bằng không.

CẦN NHỚ

Bình phương tối thiểu là một phép chiếu vuông góc. Phân rã là danh sách trực kèm hệ số kéo dẫn. Số điều kiện nói bạn sắp mất bao nhiêu chữ số có nghĩa — và trong huấn luyện, nó nói bạn sắp chờ lâu cỡ nào. Ba câu đó phủ gần hết chỗ đại số tuyến tính cần cho cả cây này. [T1]

F. Kết nối

Mục này là tiền đề trực tiếp của [A/01/02-vi-phan-va-toi-uu](#): hình dạng landscape bàn ở đó chính là phổ trị riêng của Hessian nói ở đây. Nó cũng là tiền đề của [A/01/04-hinh-hoc-va-tin-hieu](#), nơi $SO(3)$ xuất hiện như một ràng buộc trực giao mà SVD dùng để ép về. Theo chiều ngang, C dùng lại xấp xỉ hạng thấp ở phần tinh chỉnh hiệu quả tham số. Còn E dùng lại phép chiếu khi bàn *linear probe*. Một linear probe *chính là* bài bình phương tối thiểu trên không gian đặc trưng, nên nó thừa hưởng đúng số điều kiện đã bàn ở trên. Mini project cuối chương (PCA và *whitening* trên CIFAR-10) là bài kiểm tra thực hành cho toàn mục.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao giải phương trình chuẩn tắc bằng cách lập $A^T A$ làm mất khoảng một nửa số chữ số có nghĩa, trong khi QR thì không?
2. Số điều kiện lớn của một ma trận trọng số nói gì và *không* nói gì về độ ổn định huấn luyện?
3. Hessian của hiệu chỉnh chùm tia suy biến bảy chiều. Vì sao đó không phải lỗi, và làm gì thì đúng?
4. Bạn đo được 95% phương sai của đặc trưng nằm trong 10 chiều đầu. Vì sao điều đó không cho phép kết luận có thể nén đặc trưng về 10 chiều mà không mất độ chính xác?
5. Cùng là “hạng thấp”, vì sao rank collapse trong học tự giám sát là bệnh còn LoRA lại là tính năng?
6. Bạn phải chọn giữa Cholesky, QR và SVD cho một bài bình phương tối thiểu 20 nghìn phương trình, 300 ẩn, ma trận thưa và điều hoà tốt. Chọn cái nào và vì sao?

Nguồn

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

1.2 Vi phân nhiều biến và tối ưu hoá (Matrix Calculus and Optimization)

PHỤ THUỘC [A/01-toan-toi-thieu/01-dai-so-tuyen-tinh](#) (phổ trị riêng, số điều kiện — toàn bộ phần “hình dạng landscape” dưới đây là cách đọc lại phổ ấy).

TÓM TẮT

Mục này giải thích cấu trúc của phép lấy đạo hàm trên đồ thị tính toán và hình dạng của bài toán tối ưu mà huấn luyện thực sự giải. Đọc xong bạn tự derive được backward của một tầng bất kỳ mà không nhầm chiều. Bạn biết vì sao **lan truyền ngược** (*backpropagation*) đặt về bộ nhớ chứ không đặt về phép tính. Và bạn chẩn đoán được “huấn luyện chậm” thuộc loại nào trong ba loại: điều hoà tệ, learning rate sai, hay tín hiệu giám sát yếu. Nếu chỉ nhớ một điều: *khớp shape trước, dấu và hệ số sau* — và trước khi đổ lỗi cho tính không lồi, hãy kiểm tra số điều kiện, vì phần lớn “bài toán khó tối ưu” hoá ra chỉ là bài toán bị điều hoà tệ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình và tham số** — mô hình là một hàm có tham số nhận đầu vào và trả về dự đoán; tham số là các con số bên trong hàm đó mà quá trình huấn luyện được phép thay đổi.
- **hàm mất mát (loss function)** — một con số đo mức sai của dự đoán so với nhãn. Huấn luyện là quá trình đi tìm bộ tham số làm con số này nhỏ nhất.
- **gradient** — vector các đạo hàm riêng của hàm mất mát theo từng tham số; nó chỉ hướng làm loss tăng nhanh nhất, nên ta cập nhật theo hướng ngược lại.
- **learning rate** — độ dài bước mà thuật toán tối ưu đi theo hướng ngược gradient ở mỗi lần cập nhật. Quá lớn thì nghiệm bật ra ngoài và phân kỳ; quá nhỏ thì hội tụ chậm tới mức vô dụng.
- **mini-batch và epoch** — mini-batch là một nhóm nhỏ mẫu dùng để ước lượng gradient cho *một* lần cập nhật; một epoch là một lượt đi hết toàn bộ tập huấn luyện.
- **tập validation** — phần dữ liệu tách riêng, không dùng để cập nhật tham số, chỉ dùng để đo xem mô hình đang tốt lên hay xấu đi. Nó tồn tại vì loss trên tập huấn luyện luôn giảm kể cả khi mô hình đang học vẹt.
- **hàm lồi (convex)** — hàm mà đoạn thẳng nối hai điểm bất kỳ trên đồ thị luôn nằm trên đồ thị — hình dạng “lồng chảo”. Với hàm lồi, mọi cực tiểu địa phương đều là cực tiểu toàn cục; mạng nơ-ron thì không lồi.
- **Jacobian và Hessian** — Jacobian là ma trận đạo hàm bậc nhất của một hàm nhiều đầu vào nhiều đầu ra — nó là bản tuyến tính hoá tốt nhất tại một điểm. Hessian là ma trận đạo hàm bậc hai, đo độ cong.

A. Đặt vấn đề

Bạn đã viết **ma trận Jacobian** (*Jacobian*) bằng tay cho một hàm chi phí **sai số tái chiếu** (*reprojection error*) và đã từng mất một buổi chiều vì nhầm dấu hoặc nhầm thứ tự nhân. Trong deep learning, **autograd** làm hộ việc đó, nên câu hỏi đổi hình. Không còn là “đạo hàm bằng bao nhiêu”, mà là “vì sao **gradient** chết ở tầng thứ ba”, “vì sao đổi thứ tự hai phép toán làm bộ nhớ tăng gấp đôi”, và “vì sao **hàm mất mát** (*loss function*) giảm nhưng mô hình không học được gì”. Trả lời những câu đó cần hiểu **cấu trúc** của phép lấy đạo hàm, không cần kỹ năng đạo hàm tay.

Học xong mục này thì làm được gì mà trước đó không làm được? Ba việc: viết và kiểm chứng một **lượt ngược** (*backward pass*) tự cài trong mười lăm phút thay vì nửa ngày; đọc một đường cong huấn luyện và đặt tên đúng cho nguyên nhân trước khi thử ngẫu nhiên các siêu tham số; và hiểu vì sao mọi optimizer hiện đại đều là biến thể của cùng một ý tưởng, nên không cần học thuộc từng cái.

B. Bộ máy vi phân: khớp shape trước

TRỰC GIÁC

Quy tắc chuỗi ở dạng ma trận là một bài toán **lắp ống dẫn**. Mỗi tầng là một cái ống có đường kính vào và đường kính ra; Jacobian của tầng là tấm chuyển tiếp kích thước (đường kính ra) $\times$ (đường kính vào). Backprop là ghép các tấm chuyển tiếp từ cuối về đầu, và cách duy nhất để ghép sai là ghép hai tấm không khớp đường kính. Vì thế: khớp shape trước, lo dấu và hệ số sau — nếu shape khớp thì công thức chỉ còn sai được ở một hằng số hoặc một dấu trừ, và cả hai lộ ra ngay trong một lần **kiểm tra gradient** (*gradient check*).

Với hàm $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$, Jacobian $J \in \mathbb{R}^{m \times n}$ là bản tuyến tính hoá tốt nhất tại một điểm:

$$f(x + \delta) \approx f(x) + J\delta.$$

Nói bằng lời: *trong một lân cận đủ nhỏ, mọi hàm trơn đều là một phép nhân ma trận* — và đó là toàn bộ lý do vi phân hữu ích. **Quy tắc chuỗi** (*chain rule*) nói Jacobian của hàm hợp là tích các Jacobian theo đúng thứ tự hợp, $J_{g \circ f} = J_g J_f$.

Toàn bộ lan truyền ngược [8] chỉ là một nhận xét về *thứ tự nhân*. Đầu ra cuối là một số vô hướng, tức $m = 1$ ở tầng hàm mất mát. Khi đó nhân tích ấy từ trái sang phải rẻ hơn từ phải sang trái rất nhiều. Lý do: mỗi bước ta nhân một vector hàng với một ma trận, thay vì nhân hai ma trận. Đó là lý do nó tên là **reverse-mode automatic differentiation**, tức là đi ngược từ đầu ra vô hướng về từng tham số; cần phân biệt *lan truyền ngược* là thuật toán còn **lượt ngược** (*backward pass*) là một lần chạy nó. Đây cũng là lý do nó tốn bộ nhớ: muốn nhân ngược thì phải giữ lại giá trị trung gian của lượt xuôi. Đánh đổi bộ nhớ lấy tính toán ấy quay lại nguyên vẹn ở D dưới cái tên **gradient checkpointing** [9].

↔ **ràng buộc bởi** D — bộ nhớ kích hoạt do lượt xuôi để lại mới là thứ chặn batch size trên một GPU, không phải số trọng số

Phiên bản đơn giản hoá của mẹo khớp shape. Nếu $Y = XW$ với $X \in \mathbb{R}^{B \times d_{\text{in}}}$, $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$, thì gradient của một đại lượng phải có đúng shape của chính đại lượng đó. Vậy $\partial L / \partial W$ phải là $d_{\text{in}} \times d_{\text{out}}$; hai thứ có sẵn để ghép ra kích thước đó là $X^\top$ ($d_{\text{in}} \times B$) và $\partial L / \partial Y$ ($B \times d_{\text{out}}$); chỉ có một cách ghép hợp lệ, nên

$$\frac{\partial L}{\partial W} = X^\top \frac{\partial L}{\partial Y}, \quad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W^\top.$$

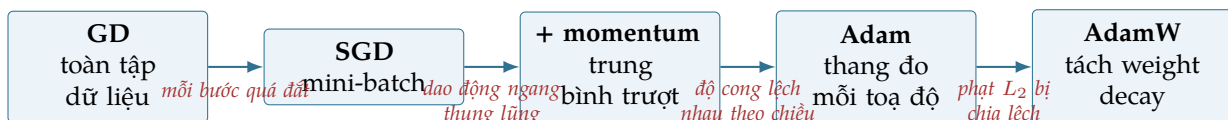
Không cần nhớ gì, chỉ cần đếm chiều. Quy tắc này giải quyết khoảng chín phần mười trường hợp thực tế; phần còn lại — chuẩn hoá, **softmax**, phép toán chia sẻ trọng số — thì kiểm bằng

`torch.autograd.gradcheck` trên đầu vào kiểu `double`, và nếu sai thì gần như luôn sai ở một chỗ tổng-hợp-theo-*batch* bị bỏ quên.

Ma trận Hessian (*Hessian*) gần như không bao giờ được lập tường minh, vì kích thước của nó là bình phương số tham số. Nhưng *tích Hessian với một vector* tính được với chi phí cỡ một lượt ngược, và thế là đủ để đo độ cong theo một hướng. Vai trò của Hessian ở đây không phải để làm bước Newton mà để *đọc bài toán*: trị riêng lớn nhất chặn *learning rate* ổn định, phổ trị riêng cho biết landscape dẹt hay nhọn, trị riêng âm cho biết ta đang ở **điểm yên ngựa** (*saddle point*) chứ không phải cực tiểu.

C. Vì sao optimizer có hình dạng như hiện nay

Hình 1.3 đọc theo trục thời gian: mỗi bước sinh ra vì bước trước để lại một nút thắt cụ thể, không phải vì ai đó muốn thêm một siêu tham số.



Hình 1.3. Dòng tiến hoá của optimizer. Nhãn dưới mỗi mũi tên là nút thắt mà bước kế tiếp gỡ; đọc theo chiều này thì không cần học thuộc từng thuật toán, chỉ cần nhớ nó sinh ra để chữa cái gì.

I Mạch 2 — cùng dòng tiến hoá, kèm giá phải trả

- 1 Vấn đề** hàm mất mát nhiều chiều, không có nghiệm đóng.

Giải pháp đi ngược hướng gradient từng bước nhỏ.

Giá định hàm khả vi và trơn đủ để tuyến tính hoá còn đúng trong lân cận cỡ learning rate.

Giá phải trả phải chọn learning rate; chọn sai thì hoặc phân kỳ hoặc bò.
- 2 Vấn đề** tính gradient trên toàn tập dữ liệu quá đắt.

Giải pháp ước lượng trên một mini-batch.

Hệ quả mỗi bước rẽ đi hàng nghìn lần nên ta đi được nhiều bước hơn với cùng ngân sách.

Vấn đề mới gradient thành biến ngẫu nhiên, với nhiều tỉ lệ nghịch với căn của **kích thước batch** (*batch size*). Tăng batch gấp bốn chỉ giảm nhiễu một nửa. Quan hệ lợi ích giảm dần đó quyết định phần lớn cách chọn batch size [10].
- 3 Vấn đề** các hướng có độ cong rất khác nhau nên một learning rate chung không vừa cho hướng nào.

Giải pháp co giãn từng tọa độ theo lịch sử độ lớn gradient, tức họ Adam [5].

Giá định độ cong xấp xỉ tách rời được theo từng tọa độ.

Giá phải trả mất đảm bảo hội tụ của SGD trong vài trường hợp, và sinh tương tác lạ với *weight decay* — chính chỗ đó đẻ ra AdamW [11].
- 4 Vấn đề** nhiều mini-batch làm quỹ đạo dao động ở cuối quá trình.

Giải pháp giảm learning rate theo lịch trình.

Hệ quả bước cuối đủ nhỏ để trung bình hoá nhiễu.

Vấn đề mới lịch trình thành một siêu tham số nhạy, tương tác với cả batch size lẫn số epoch — nên chép kiến trúc mà không chép lịch trình thì không tái lập được kết quả.

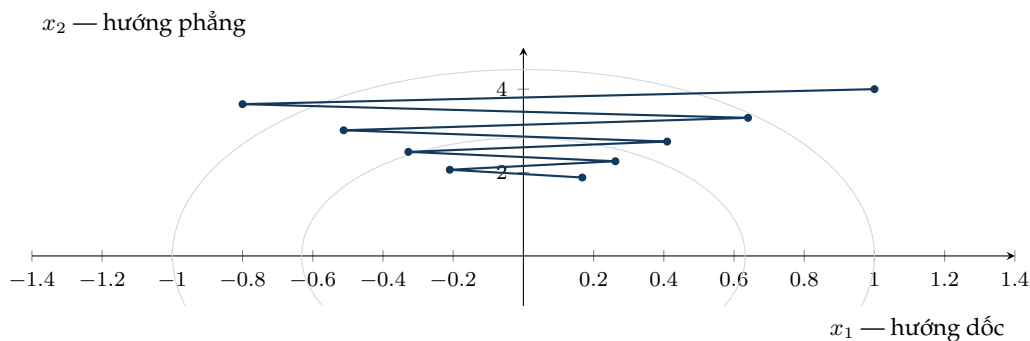
Bảng 1.2. Bốn optimizer thường gặp trên cùng tiêu chí. Cột cuối quan trọng nhất.

Thuật toán	Cơ chế	Mạnh khi nào	Hỏng khi nào
SGD	Bước theo gradient mini-batch	Bài toán điều hoà tốt sau chuẩn hoá; hay tổng quát hoá tốt hơn trên thị giác	Landscape lệch thang đo mạnh; cần rất nhiều công tune learning rate
SGD + momentum	Trung bình trượt gradient	Thung lũng hẹp dài — triệt tiêu dao động ngang	Momentum quá lớn gây vọt lố ở giai đoạn cuối
Adam	Chia mỗi toạ độ cho ước lượng độ lớn riêng	Gradient thưa hoặc lệch thang đo mạnh, ví dụ <i>transformer</i>	Đầu quá trình khi ước lượng phương sai chưa ổn (cần <i>warmup</i>); tổng quát hoá đôi khi kém hơn SGD
AdamW	Như Adam, tách weight decay khỏi bước thích nghi	Mặc định hợp lý cho hầu hết mô hình lớn hiện nay	Vẫn cần warmup và lịch trình; không cứu được bài toán mà tín hiệu giám sát sai

D. Lỗi hay không lỗi, và hình dạng thật của landscape

Câu nói phổ thông “deep learning khó vì hàm mất mát không lồi” đúng về chữ nhưng dẫn tới trực giác sai. Với bài toán **lồi** (*convex*), mọi cực tiểu địa phương là cực tiểu toàn cục và ta có chứng nhận dừng. Với bài toán không lồi cả hai thứ đó mất. Nhưng thực nghiệm trên mạng lớn cho thấy thủ phạm làm chậm không phải các cực tiểu địa phương tồi: trong không gian rất nhiều chiều, một điểm dừng ngẫu nhiên gần như chắc chắn là điểm yên ngựa, vì để là cực tiểu thì hàng triệu trị riêng phải dương *cùng lúc*. Cái làm chậm là vùng phẳng quanh yên ngựa và vùng có số điều kiện tệ — tức đúng hiện tượng ở mục trước.

Ví dụ nhỏ nhất tính tay được, và nó đủ giải thích cả hiện tượng. Với $L(x) = \frac{1}{2}(x_1^2 + \varepsilon x_2^2)$, hạ gradient bước η nhân thành phần thứ nhất với $(1 - \eta)$ và thành phần thứ hai với $(1 - \eta\varepsilon)$ mỗi vòng. Ổn định buộc $\eta < 2$. Lấy $\eta = 1,8$ và $\varepsilon = 0,05$. Hướng thứ nhất dao động đổi dấu mỗi bước với hệ số $-0,8$; hướng thứ hai mỗi vòng chỉ co 9%. Tỉ số $1/\varepsilon = 20$ chính là số điều kiện. Hình 1.4 vẽ đúng quỹ đạo đó: chi phí không nằm ở chỗ “đi xuống”, nó nằm ở chỗ đi ngang thung lũng.



Hình 1.4. Hạ gradient trên hàm toàn phương có số điều kiện 20: quỹ đạo nảy qua nảy lại theo hướng dốc trong khi bò rất chậm theo hướng phẳng. Chuẩn hoá đầu vào và các optimizer thích nghi đều nhắm vào việc làm hai trục của ellipse gần bằng nhau, tức là làm hình này tròn lại.

Khác biệt thứ hai, thực dụng hơn: khi không lồi, *cực tiểu nào* ta rơi vào là một phần của kết quả. Có bằng chứng thực nghiệm rằng nghiệm ở vùng dẹt tổng quát hoá tốt hơn nghiệm ở vùng nhọn, và batch lớn có xu hướng rơi vào vùng nhọn nếu không hiệu chỉnh gì [12]. Đây là quan sát đo

được và tái lập nhiều lần, nhưng cách giải thích vẫn còn tranh cãi và định nghĩa “dệt” phụ thuộc cách tham số hoá — hãy nhớ ở mức hiện tượng, đừng nhớ ở mức định lý.

E. Điều kiện dừng nào có nghĩa

Trong tối ưu cổ điển ta dừng khi chuẩn gradient nhỏ. Trong huấn luyện mạng, tiêu chí đó gần như vô dụng: gradient mini-batch không bao giờ nhỏ vì nhiễu lấy mẫu, và ngay cả gradient toàn phần nhỏ cũng không nói gì về chất lượng mô hình. Thứ thực sự được dùng là dừng theo chỉ số trên tập *validation* — tức dừng khi mục tiêu ta *quan tâm*, không phải mục tiêu ta *tối ưu*, ngừng cải thiện.

→ **tiền đề A/01** — câu hỏi “validation đã cải thiện chưa” là câu hỏi thống kê về sai số lấy mẫu, được trả lời ở mục xác suất kế tiếp

Chuyển từ “điều kiện dừng của thuật toán” sang “điều kiện dừng của phép ước lượng ngoài mẫu” là một dịch chuyển tư duy lớn. Nó cũng là cầu nối sang **A/01/03**. Câu hỏi “validation đã ngừng cải thiện chưa” là câu hỏi thống kê, không phải câu hỏi giải tích.

F. Giới hạn và trường hợp hỏng

- **Giả định khả vi bị vi phạm thường xuyên hơn ta tưởng.** *ReLU* không khả vi tại không (vô hại, tập điểm có độ đo không), nhưng các phép toán rời rạc thật thì không cứu được: bước lọc bỏ các hộp trùng nhau và chỉ giữ hộp điểm cao nhất (*non-maximum suppression*) dùng *argmax*, phép lấy mẫu cứng, chỉ số nguyên trong **lượng tử hoá** (*quantization*) đều làm gradient bằng không ở mọi nơi. Cả một mảng kỹ thuật ở C và D — *surrogate loss, straight-through estimator, Gumbel-softmax* — tồn tại chỉ để vá chỗ này, và mỗi cách vá đưa vào một thiên lệch riêng.
- **hệ quả của C** — mọi cách vá chỗ không khả vi đều sinh ra từ đúng một sự thật: gradient bằng không ở mọi nơi trên một phép toán rời rạc
- **Autograd không cứu được sai số số học.** Gradient có thể đúng công thức nhưng vẫn tràn hoặc mất chính xác ở *half precision*. Triệu chứng là loss thành NaN sau vài trăm bước. Đó là lý do tồn tại của *loss scaling* trong huấn luyện hỗn hợp độ chính xác [13]. Ở chiều ngược lại, gradient check với float32 hay báo sai giả; phải kiểm ở double.
- **Đừng suy từ bài toán lồi sang mạng sâu.** Trực giác “quỹ đạo trơn tru đi xuống lòng chảo” mô tả sai cái đang xảy ra: quỹ đạo thật đi qua vùng có độ cong đổi dấu, và learning rate lớn ở giai đoạn đầu đóng vai trò **chính quy hoá** (*regularization*) chứ không chỉ là “đi nhanh” — giảm nó quá sớm làm mất tác dụng đó.

CẠM BÃY

Loss giảm không có nghĩa mô hình đang học thứ bạn muốn. Ba trường hợp hay gặp và cách phân biệt: loss giảm vì mô hình học thiên lệch của nhãn (kiểm bằng cách so với *baseline* đoán lớp phổ biến nhất); loss train giảm mà validation đi ngang vì **rò rỉ dữ liệu** (*data leakage*) làm hai tập không độc lập (kiểm bằng cách chia theo nhóm nguồn); loss giảm đều nhưng chỉ số nghiệp vụ đứng yên vì hàm mất mát không đại diện tốt cho chỉ số đó (kiểm bằng cách vẽ tương quan giữa hai đại lượng). Luôn xem loss như một *proxy* có sai lệch; B/09 mổ xẻ sai lệch ấy.

CẦN NHỚ

Khớp shape trước, dấu và hệ số sau. Trước khi đổ lỗi cho tính không lồi, hãy kiểm tra số điều kiện. Và đừng bao giờ dừng huấn luyện theo chuẩn gradient — dừng theo chỉ số trên tập validation, vì đó mới là đại lượng bạn thật sự quan tâm. [T1]

G. Kết nối

Mục này nhận trực tiếp từ [A/01/01-dai-so-tuyen-tinh](#) (Hessian và phổ của nó) và chuyển tiếp sang [A/01/03-xac-suat-va-thong-tin](#), nơi câu hỏi “validation ngừng cải thiện chưa” được trả lời bằng công cụ thống kê thay vì bằng mắt. Theo chiều ngang, đây là tiền đề của hai chỗ. Thứ nhất là toàn bộ chương optimizer và lịch trình learning rate ở D. Thứ hai là phần thiết kế hàm mất mát ở B/09, nơi ta thấy nhiều lựa chọn loss thật ra là lựa chọn về *hình dạng gradient* chứ không về ý nghĩa thống kê.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao chế độ ngược tổn bộ nhớ còn chế độ xuôi thì không, và điều đó dẫn tới kỹ thuật gì khi mô hình quá lớn?
2. Bạn derive được $\partial L / \partial W$ mà không cần nhớ công thức. Lập luận đó dựa trên tính chất nào, và khi nào nó không đủ để xác định duy nhất đáp án?
3. Trong không gian nhiều triệu chiều, vì sao điểm dừng gần như chắc chắn là yên ngựa chứ không phải cực tiểu địa phương?
4. Vì sao “chuẩn gradient nhỏ” là điều kiện dừng vô nghĩa khi huấn luyện với mini-batch?
5. Tăng batch size gấp bốn thì nhiễu gradient giảm bao nhiêu, và vì sao quan hệ đó khiến batch rất lớn kém hiệu quả về chi phí?
6. Loss giảm ổn định suốt 50 epoch nhưng mô hình vô dụng khi triển khai — ba nguyên nhân nào giải thích được chuyện đó, và mỗi nguyên nhân phân biệt bằng phép kiểm nào?

Nguồn

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

1.3 Xác suất, độ hợp lý và lý thuyết thông tin (Probability, Likelihood and Information Theory)

PHỤ THUỘC A/01-toan-toi-thieu/02-vi-phan-va-toi-uu (điều kiện dùng theo validation — mục này cung cấp công cụ để trả lời “validation đã thật sự cải thiện chưa”).

TÓM TẮT

Mục này làm hai việc: cho thấy gần như mọi **hàm mất mát** (*loss function*) là âm log **độ hợp lý** (*likelihood*) của một giả định phân phối được chọn ngầm, và cho bạn công cụ để biết một con số đo được đáng tin đến đâu. Đọc xong bạn không cần nhớ công thức loss nào nữa, vì suy ra được từ giả định. Bạn biết ngay loss đó hỏng ở đâu: đúng chỗ giả định vỡ. Và bạn tính được trong ba mươi giây xem chênh lệch một điểm phần trăm giữa hai mô hình có đáng bàn không. Nếu chỉ nhớ một điều: *mọi loss là một giả định phân phối trá hình, và mọi con số đo được là một ước lượng có sai số* — ai bỏ qua về thứ hai sẽ dành hàng tháng để đuổi theo nhiều.

KIẾN THỨC CẦN CÓ TRƯỚC

- **biến ngẫu nhiên, kỳ vọng, phương sai** — biến ngẫu nhiên là một đại lượng mà giá trị phụ thuộc kết quả của một phép thử; kỳ vọng là giá trị trung bình dài hạn của nó; phương sai đo mức dao động quanh kỳ vọng, và căn bậc hai của phương sai là độ lệch chuẩn.
- **phân phối (distribution)** — quy luật gán xác suất cho các giá trị mà biến ngẫu nhiên có thể nhận. Chọn một phân phối là đưa ra một giả định về cách dữ liệu được sinh ra.
- **độ hợp lý (likelihood)** — với một bộ tham số cho trước, xác suất mà mô hình gán cho *dữ liệu đã quan sát được*. Lưu ý nó là hàm của tham số, không phải hàm của dữ liệu.
- **prior** — niềm tin về tham số *trước khi* nhìn dữ liệu, viết dưới dạng một phân phối. Trong thực hành, một prior tương đương với một số hạng phạt cộng vào hàm mất mát.
- **hàm mất mát (loss function)** — một con số đo mức sai của dự đoán so với nhãn; huấn luyện là đi tìm tham số làm nó nhỏ nhất.
- **softmax** — phép biến một vector số thực bất kỳ thành một bộ số dương cộng lại bằng một, để đọc như xác suất của từng lớp. “Đọc như” không có nghĩa “là”.
- **tập test và tập validation** — validation là phần dữ liệu dùng để chọn siêu tham số và quyết định khi nào dừng; test là phần chỉ được chạm vào *một lần* ở cuối để báo cáo. Dùng test như validation làm mọi con số trở nên lạc quan giả.
- **độ chính xác (accuracy)** — tỉ lệ mẫu được dự đoán đúng. Nó là trung bình của các biến chỉ báo đúng hay sai, nên bản thân nó cũng là một ước lượng có sai số.

A. Đặt vấn đề

Hai câu nói kết thúc rất nhiều tranh luận kỹ thuật một cách sai: “mô hình A đạt 90,2%, mô hình B đạt 89,1%, vậy A tốt hơn” và “loss của tôi là **entropy chéo** (*cross-entropy*) vì mọi người dùng

cross-entropy”. Câu thứ nhất là một phát biểu thống kê không kèm sai số. Câu thứ hai là một lựa chọn mô hình xác suất được nhớ như một quy ước. Mục này chứa cả hai bằng cùng một bộ công cụ.

Học xong mục này thì làm được gì mà trước đó không làm được? Bốn việc: suy ra hàm mất mát từ giả định về nhiễu thay vì tra bảng; dự đoán trước loss sẽ hỏng ở đâu; tính khoảng tin cậy và quyết định một thí nghiệm có đủ mẫu không *trước khi* chạy; và đọc đúng hướng của **phân kỳ KL** (*Kullback–Leibler divergence*) trong một paper để biết mô hình sẽ trải mỏng hay chọn một mode. Nền lý thuyết ở [14], [15]; phân thông tin ở [16].

TRỰC GIÁC

Cross-entropy là hoá đơn tiền điện của việc mã hoá sự thật bằng bảng mã của mô hình. Mô hình tin chắc vào điều đúng thì sự kiện thật rơi vào một ký hiệu ngắn và hoá đơn nhỏ; mô hình tin chắc vào điều sai thì sự kiện thật rơi vào một ký hiệu dài kinh khủng, đúng bằng $-\log q$ của một xác suất gần không. *Entropy* là hoá đơn tối thiểu không tránh được; KL là phần phụ trội phải trả vì dùng sai bảng mã. Cách nhìn này giải thích ngay vì sao mô hình bị phạt nặng bất đối xứng khi *tự tin mà sai*, chứ không phải khi chỉ đơn giản là sai.

B. Sai số lấy mẫu: một tập test 500 mẫu đáng tin đến đâu

Độ chính xác đo trên tập test là trung bình của các biến Bernoulli, nên sai số chuẩn là $\sqrt{p(1-p)/n}$. Nói bằng lời: *đại lượng này đo mức dao động của con số nếu bạn rút một mẫu 500 phần tử khác từ cùng phân phối* — không hơn.

Tính tay. Với $n = 500$, $p = 0,90$: sai số chuẩn $= \sqrt{0,9 \cdot 0,1/500} = \sqrt{0,00018} \approx 0,0134$, tức 1,34 điểm phần trăm. Khoảng tin cậy 95% xấp xỉ $90\% \pm 2,6\%$, tức từ 87,4% tới 92,6% — rộng hơn năm điểm.

Ở đây có một tình thế quan trọng, và bỏ qua nó thì sai theo cả hai chiều. Khi hai mô hình được đánh giá trên *cùng* một tập test, chúng sai ở phần lớn cùng những mẫu khó, nên phép so sánh đúng là so sánh *ghép cặp*: cái cần đếm là số mẫu hai mô hình bất đồng, và độ bất định của hiệu số nhỏ hơn nhiều so với độ bất định của từng con số riêng lẻ. Nếu trong 500 mẫu chỉ có 20 mẫu bất đồng và 15 nghiêng về A thì kết luận còn yếu; nếu 60 mẫu bất đồng và 45 nghiêng về A thì hiệu số một điểm có thể có ý nghĩa. Công cụ đúng là kiểm định ghép cặp (McNemar) hoặc *bootstrap* trên chính tập test, không phải hai khoảng tin cậy vẽ cạnh nhau [17].

Còn hai nguồn bất định nữa mà công thức trên *không* bắt được, và chúng thường lớn hơn:

- **Bất định do chọn tập test.** Công thức chỉ trả lời “nếu rút 500 mẫu khác từ cùng phân phối”. Dòng công trình dựng lại tập test của ImageNet theo đúng quy trình gốc cho thấy độ chính xác tụt hàng điểm [18] — tức bản thân quy trình thu thập cũng là một nguồn ngẫu nhiên.
- **Nhãn sai.** Các tập chuẩn phổ biến có tỉ lệ nhãn sai đủ để đảo thứ hạng ở phần đỉnh bảng [19]. Nó đặt một trần cứng lên mọi phép so sánh, và không phương pháp thống kê nào vượt qua được trần đó.

C. Likelihood, MLE, MAP: nguồn gốc chung của mọi hàm mất mát

Giả sử ta mô hình hoá dữ liệu bằng một phân phối có tham số θ . **Ước lượng độ hợp lý cực đại** (*maximum likelihood estimation, MLE*) chọn θ cực đại $\prod_i p(y_i | x_i; \theta)$, tương đương cực tiểu $-\sum_i \log p(y_i | x_i; \theta)$. Nói bằng lời: *huấn luyện chính là tìm bộ tham số làm cho dữ liệu quan sát được trở nên ít bất ngờ nhất*. Chọn dạng phân phối nào thì ra loss này — và đó là toàn bộ nội dung Bảng 1.3.

Bảng 1.3. Hàm mất mát nào cũng là một giả định phân phối. Cột cuối là cột đáng giá nhất: giả định vỡ ở đâu thì loss hỏng ở đó.

Loss	Giả định phân phối	Cho ra ước lượng gì	Hỏng khi nào
MSE	Gaussian, phương sai hằng số	Trung bình có điều kiện	Có ngoại lai; nhiều phụ thuộc tín hiệu; mục tiêu đa phương thức thì cho ra trung bình mờ
L1	Laplace	Trung vị có điều kiện	Cần ước lượng trung bình; gradient hằng số gây dao động gần nghiệm
Cross-entropy	Categorical	Xác suất từng lớp	Nhân sai hoặc mất cân bằng cực đoan; phạt rất nặng khi tự tin mà sai
Phạt L_2	Cực đại hậu nghiệm (<i>maximum a posteriori</i> , MAP) với prior Gaussian	Nghiệm co về 0	Khi thang đo các tham số khác nhau nhiều

Bảng trên giải thích một chuyện hay gây bối rối: vì sao hồi quy độ sâu bằng MSE cho ra biên vật thể mờ. Ở biên, độ sâu thật là *đa phương thức*: hoặc thuộc vật trước, hoặc thuộc nền. MLE dưới giả định Gaussian đơn phương thức buộc mô hình trả về trung bình của hai đáp án đúng. Kết quả là một giá trị sai nằm ở giữa. Không siêu tham số nào cứu được; phải đổi giả định phân phối, và đó chính là động cơ của các mô hình sinh có điều kiện ở C.

→ **tiền đề** B/09 — mọi lựa chọn hàm mất mát ở chương thiết kế loss bắt đầu bằng việc chọn — có ý thức hay không — một giả định phân phối

MAP thêm một prior: cực đại $p(\theta \mid \text{dữ liệu}) \propto p(\text{dữ liệu} \mid \theta) p(\theta)$, tương đương thêm một số hạng phạt vào loss. Với nền SLAM đây là chỗ nổi quen thuộc: một prior trên pose *chính là* một số hạng phạt trong hàm chi phí, và damping của Levenberg–Marquardt hoạt động y hệt một prior Gaussian quanh điểm hiện tại — cùng một cơ chế, hai cái tên.

Cần trung thực một chỗ: *không phải mọi loss đều là âm log độ hợp lý*. **Focal loss** [20] là phép định lại trọng số theo độ khó; **làm mượt nhãn** (*label smoothing*) [21] cố ý làm sai lệch mục tiêu; các loss dạng biên (triplet, contrastive) tối ưu một thứ tự chứ không một mật độ. Đó là kỹ thuật, không phải hệ quả của một mô hình sinh — và chính vì thế xác suất mà mô hình xuất ra sau khi huấn luyện bằng chúng không còn được **hiệu chuẩn** (*calibrated*).

D. Entropy, cross-entropy, KL và sự bất đối xứng

Ba đại lượng, một quan hệ:

$$H(p) = - \sum_i p_i \log p_i, \quad H(p, q) = - \sum_i p_i \log q_i, \quad D_{\text{KL}}(p \parallel q) = H(p, q) - H(p).$$

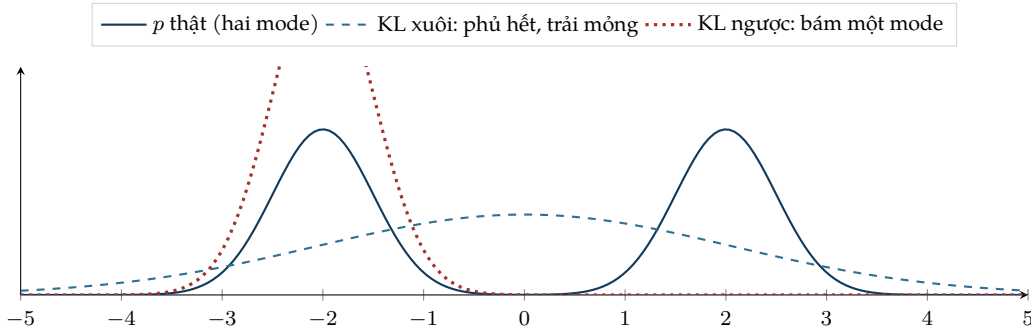
Vì $H(p)$ không phụ thuộc tham số mô hình, *cực tiểu cross-entropy chính là cực tiểu KL từ dữ liệu tới mô hình, và cũng chính là MLE* — ba cách nói của một việc.

Một phép kiểm tra rẻ tiền rơi ra từ đây. Với bài toán 1000 lớp, mô hình khởi tạo hợp lý phải cho loss ban đầu xấp xỉ $\ln 1000 \approx 6,91$. Nếu là 20 thì khởi tạo hoặc tầng cuối sai; nếu là 2 thì có rò rỉ nhãn. Một dòng kiểm tra, bắt được cả một lớp bug.

KL không đối xứng, và đây không phải khiếm khuyết toán học mà là một lựa chọn về hành vi. Ví dụ tính tay với hai phân phối Bernoulli $p = (0,5; 0,5)$ và $q = (0,9; 0,1)$:

$$D_{\text{KL}}(p||q) = 0,5 \ln \frac{0,5}{0,9} + 0,5 \ln \frac{0,5}{0,1} \approx 0,511, \quad D_{\text{KL}}(q||p) = 0,9 \ln \frac{0,9}{0,5} + 0,1 \ln \frac{0,1}{0,5} \approx 0,368.$$

Hai con số khác nhau vì hai đại lượng phạt hai kiểu sai lầm khác nhau. Hình 1.5 cho thấy hệ quả rõ hơn mọi lời giải thích: cùng một phân phối thật hai đỉnh, hai hướng KL cho ra hai nghiệm hoàn toàn khác.



Hình 1.5. Hai hướng của KL cho hai hành vi trái ngược khi mô hình quá đơn giản để khớp phân phối thật. KL xuôi $D_{\text{KL}}(p||q)$ phạt nặng chỗ p có khối lượng mà q gần bằng không, nên nghiệm bị ép phủ cả hai mode và đặt khối lượng vào vùng trống ở giữa. KL ngược $D_{\text{KL}}(q||p)$ phạt chỗ q đặt khối lượng mà p không có, nên nghiệm chọn một mode và bỏ mode kia.

Sự khác biệt ấy giải thích ba hiện tượng sẽ gặp lại:

↪ cùng nguyên lý với C — chiều của KL được tối ưu quyết định mô hình sinh trái mỏng qua mọi mode hay bám lấy một mode — cùng cơ chế ở VAE, chưng cất và huấn luyện theo độ hợp lý

- **Mô hình sinh huấn luyện theo độ hợp lý sinh ra mẫu mờ.** MLE dùng KL xuôi, nên mô hình buộc phải phủ cả những vùng trống giữa các mode. Mẫu rút từ vùng ấy không giống bất kỳ mẫu thật nào.
- **Posterior collapse trong VAE [22]** — số hạng KL ngược tới prior khiến mô hình thà bỏ hẳn vài chiều tiềm ẩn còn hơn trái mỏng.
- **“Dark knowledge” trong chưng cất tri thức (knowledge distillation) [23]** — học trò khớp phân phối thầy theo chiều xuôi, nên bị buộc phải phủ cả các đuôi xác suất của những lớp không phải đáp án; chính các đuôi đó mang thông tin về quan hệ giữa các lớp.

E. Giới hạn và trường hợp hỏng

- **Giả định độc lập và cùng phân phối gần như luôn sai trong thị giác.** Các khung hình liên tiếp của một video giống nhau; nhiều ảnh của cùng một bệnh nhân cũng vậy; nhiều mẫu chụp cùng buổi thì cùng ánh sáng. Một tập test 500 ảnh trích từ 20 video có kích thước mẫu hiệu dụng gần 20 hơn gần 500. Khoảng tin cậy thật vì thế rộng gấp nhiều lần. Chia tập theo nhóm (theo video, theo người, theo thiết bị) quan trọng hơn mọi tinh chỉnh mô hình.
- **Xác suất mà mạng xuất ra không phải xác suất thật.** Mạng hiện đại có xu hướng quá tự tin: trong nhóm dự đoán độ tin cậy 0,99, tỉ lệ đúng thực tế thấp hơn đáng kể, và điều này được đo lặp lại nhiều lần [24].

↪ hệ quả của E — mạng quá tự tin không phải lỗi cài đặt mà là hệ quả của việc tối ưu cross-entropy chứ không tối ưu độ hiệu chuẩn

Đừng dùng softmax làm ngưỡng ra quyết định khi chưa hiệu chuẩn; E bàn temperature scaling, ensemble và conformal prediction [25] như các cách vá với chi phí khác nhau.

- **Khung độ hợp lý có giới hạn khái niệm.** Nó tối ưu độ khớp phân phối, còn thứ ta quan tâm thường là một chỉ số nghiệp vụ bất đối xứng — bỏ sót một người đi bộ tệ hơn nhiều so với một báo động giả. Không prior nào biến MLE thành thứ tối ưu chỉ số đó; phải đặt vào hàm chi phí một cách tường minh, và đó là nội dung của B/09.

CẠM BẤY

“Cải thiện 0,3% trên tập test” gần như luôn là nhiều — nhưng cách sửa phổ biến, chạy nhiều seed rồi báo trung bình, *vẫn* chưa đủ nếu bạn đã dùng chính tập test đó để chọn siêu tham số. Khi tập test bị nhìn nhiều lần, nó biến thành tập validation và mọi khoảng tin cậy tính trên nó đều quá lạc quan. Đây là dạng rò rỉ tinh vi nhất, vì không có dòng code nào sai cả.

CẦN NHỚ

Trước khi hỏi “mô hình nào tốt hơn”, hãy tính $\sqrt{p(1-p)/n}$ — ba mươi giây, và nó loại bỏ phần lớn tranh luận vô ích. Trước khi chọn loss, hãy hỏi “tôi đang giả định nhiều phân phối gì” — vì loss sẽ hỏng đúng chỗ giả định đó vỡ. [T1]

F. Kết nối

Mục này khép lại phần “ngôn ngữ chung” của chương, và là tiền đề của ba nhánh xa nhau. B/09 về thiết kế hàm mất mát, nơi ta cố ý lệch khỏi độ hợp lý. C về mô hình sinh, nơi hướng của KL quyết định hành vi mẫu sinh ra. Và toàn bộ E về đánh giá và độ tin cậy. Theo chiều đối lập, nó bổ khuyết cho A/01/02: mục kia nói gradient descent đi tới đâu, mục này nói làm sao biết chỗ đó có thật sự tốt hơn hay không.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao KL không đối xứng, và điều đó thay đổi hành vi cụ thể nào của một mô hình sinh?
2. Với 500 mẫu test và độ chính xác 90%, khoảng tin cậy 95% rộng bao nhiêu? Vì sao con số đó *không* trực tiếp bác bỏ được chênh lệch một điểm giữa hai mô hình đánh giá trên cùng tập test?
3. Tập test gồm 500 khung hình trích từ 20 video. Kích thước mẫu hiệu dụng gần con số nào hơn, và hệ quả với cách chia dữ liệu là gì?
4. Vì sao hồi quy độ sâu bằng MSE cho ra biên mờ, và vì sao đối optimizer không sửa được?
5. Loss ban đầu của bài 1000 lớp phải xấp xỉ bao nhiêu, và nếu nó bằng 2 thì bạn nghi ngờ điều gì?
6. Focal loss và label smoothing không phải âm log độ hợp lý của mô hình nào. Điều đó gây hệ quả gì cho việc đọc đầu ra softmax?

Nguồn

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

1.4 Hình học biến đổi và xử lý tín hiệu (Transformation Geometry and Signal Processing)

PHỤ THUỘC [A/01-toan-toi-thieu/01-dai-so-tuyen-tinh](#) (SVD — dùng để chiếu một ma trận nhiều về $SO(3)$). Mục này là tiền đề bắt buộc của cả chương [A/02-vat-ly-tao-anh](#).

TÓM TẮT

Mục này gộp hai chủ đề vì chúng chia nhau một vai trò: cả hai mô tả cấu trúc mà dữ liệu ảnh *đã có sẵn* trước khi mô hình nhìn thấy nó. Đọc xong bạn biết vì sao một mạng dự đoán phép quay bằng bốn số khó huấn luyện hơn bằng sáu số. Bạn biết vì sao CNN không thật sự bất biến tịnh tiến. Và bạn biết vì sao một dòng `resize` sai trong dataloader làm mất vài điểm độ chính xác, mà không sinh ra exception nào. Nếu chỉ nhớ một điều, hãy nhớ hai câu hỏi phòng vệ. *Trước mọi lần giảm độ phân giải: đã lọc thông thấp chưa? Trước mọi lần cho mạng xuất ra phép quay: biểu diễn này có liên tục không?* Hai câu đó bắt được phần lớn lỗi im lặng ở khâu hình học và tín hiệu.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tư thế (pose) và $SE(3)$** — tư thế là vị trí cộng hướng của một vật cứng trong không gian; tập mọi tư thế tạo thành nhóm $SE(3)$, còn tập mọi phép quay thuần túy là $SO(3)$, biểu diễn bằng ma trận 3×3 trực giao có định thức bằng một.
- **đa tạp (manifold)** — một tập hợp mà quanh mỗi điểm nó “trông giống” một không gian phẳng nhiều chiều, nhưng toàn cục thì cong. Hệ quả thực hành: không cộng trực tiếp hai phần tử được, phải cộng ở không gian tiếp tuyến.
- **convolution và kernel** — kernel là một cửa sổ nhỏ chứa các trọng số; convolution là trượt cửa sổ đó khắp ảnh và tại mỗi vị trí lấy tổng có trọng số của vùng lân cận. Đây là phép toán nền của mạng CNN.
- **CNN** — mạng nơ-ron mà phần lớn các tầng là convolution; nó được thiết kế dựa trên giả định rằng cấu trúc ảnh mang tính cục bộ và không đổi khi dịch chuyển.
- **stride và pooling** — stride là bước nhảy khi trượt kernel — stride 2 nghĩa là chỉ lấy một trên hai vị trí, tức giảm độ phân giải một nửa. Pooling cũng giảm độ phân giải bằng cách lấy giá trị lớn nhất hoặc trung bình của một vùng.
- **tần số không gian** — đo bằng “chu kỳ trên pixel”: một hoa văn 0,25 chu kỳ/pixel lặp lại sau mỗi bốn pixel. Tần số cao ứng với chi tiết mảnh, tần số thấp ứng với cấu trúc lớn.
- **biến đổi Fourier** — cách viết lại một tín hiệu thành tổng các sóng sin ở nhiều tần số khác nhau, để nhìn xem tín hiệu chứa những tần số nào và mạnh yếu ra sao.
- **nội suy (interpolation)** — khi thay đổi kích thước hoặc xoay ảnh, giá trị pixel mới phải được ước lượng từ các pixel cũ lân cận; cách ước lượng đó gọi là nội suy, và mỗi lần nội suy là một lần làm mờ thêm.

A. Đặt vấn đề

Bỏ qua cấu trúc có sẵn của dữ liệu ảnh thì mạng phải học lại nó từ đầu, tốn nhân và tốn tham số; hiểu sai cấu trúc ấy thì bạn tự tay phá dữ liệu ở khâu tiền xử lý và không bao giờ tìm ra nguyên nhân. Với người đã làm SLAM, nửa đầu mục này là ôn tập nhanh có đối trọng tâm: câu hỏi không còn là “ $SE(3)$ hoạt động thế nào” mà là “vì sao mạng khó xuất ra một phép quay”. Nửa sau thì hầu như luôn là chỗ hỏng thật, kể cả với người có kinh nghiệm.

Học xong mục này thì làm được gì mà trước đó không làm được? Chọn đúng biểu diễn đầu ra cho mọi bài toán ước lượng tư thế bằng mạng; kiểm tra một pipeline tiền xử lý và chỉ ra chỗ đang sinh hoa văn giả; và giải thích bằng ngôn ngữ tần số vì sao một mô hình huấn luyện ở 224×224 sọt điểm khi đưa vào ảnh độ phân giải khác.

TRỰC GIÁC

Aliasing không phải chuyện làm ảnh xấu đi; nó là chuyện làm ảnh *nói dối*. Nan hoa bánh xe quay ngược trong phim là ví dụ thuần khiết: máy quay không làm mờ bánh xe, nó tạo ra một chuyển động không hề tồn tại. Khi bạn resize ảnh bằng *nearest-neighbor* rồi đưa vào mạng, mạng nhận được những hoa văn cũng không hề có trên vật thật. Nó học các hoa văn ấy một cách hoàn toàn nghiêm túc. Rồi camera triển khai có độ phân giải khác, hoa văn giả đổi kiểu, và mô hình sập — mà không ai chạm vào trọng số nào.

B. Hình học: ba thao tác là đủ

Toạ độ đồng nhất (*homogeneous coordinates*) tồn tại vì một lý do thực dụng: nó biến phép tịnh tiến, vốn là phép cộng, thành phép nhân ma trận. Nhờ đó cả chuỗi biến đổi gộp được thành một tích duy nhất. Phép chiếu phối cảnh cũng viết được ở dạng tuyến tính, rồi chia cho thành phần cuối. Toàn bộ hình học nhiều view ở A/02/03 xây trên tiện lợi đó.

Nhóm quay $SO(3)$ và nhóm chuyển động cứng $SE(3)$ là các **đa tạp** (*manifold*) chứ không phải không gian vector: cộng hai ma trận quay không cho ra ma trận quay. Hệ quả thực hành — và là toàn bộ phần **đại số Lie** (*Lie algebra*) bạn cần — gói trong ba thao tác:

1. $\exp(\delta)$ đưa một vector ba chiều (trục nhân góc) thành một ma trận quay;
2. $\log(R)$ làm ngược lại, cho ra “sai lệch quay” ở dạng vector cộng được;
3. cập nhật $R \leftarrow R \exp(\delta)$ thay cho $R \leftarrow R + \delta$, tức là *cộng ở không gian tiếp tuyến rồi mới quay về đa tạp*.

Nếu bạn đã viết một solver **hiệu chỉnh chùm tia** (*bundle adjustment*) thì bạn đã cài đúng ba thao tác này; điều đáng nói là chúng không biến mất khi chuyển sang mạng nơ-ron, chỉ đổi chỗ. Nền lý thuyết ở mức kỹ sư: [26], [27].

Chỗ chúng xuất hiện lại là bài toán cho mạng *xuất ra* một phép quay, và đây là một trong ít chỗ mà lý thuyết cho lời khuyên rất rõ. Không có phép tham số hoá liên tục nào của $SO(3)$ bằng ba hoặc bốn số mà không có điểm kỳ dị hoặc điểm gián đoạn; mà một mạng liên tục theo đầu vào thì không thể khớp một hàm mục tiêu gián đoạn, nên nó cứ xử tệ đúng quanh vùng gián đoạn. Bảng 1.4 so bốn lựa chọn.

Bảng 1.4. Bốn cách cho mạng xuất ra một phép quay. Cột cuối quyết định lựa chọn.

Biểu diễn	Số đầu ra	Cơ chế đưa về $SO(3)$	Hỏng khi nào
Góc Euler	3	Nhân ba phép quay trực	<i>Gimbal lock</i> ; gián đoạn ở biên $\pm\pi$; trung bình vô nghĩa
Quaternion	4	Chuẩn hoá về chuẩn 1	Phủ hai lần (q và $-q$ cùng một phép quay) nên hàm mục tiêu gián đoạn; phải xử lý dấu
6D (hai vector)	6	Gram–Schmidt trực giao hoá	Không có điểm gián đoạn — lựa chọn mặc định hiện nay; chỉ tốn thêm vài phép tính
Ma trận 9 số	9	Chiếu về $SO(3)$ bằng SVD	Cũng liên tục, nhưng SVD trong vòng huấn luyện đắt hơn và đạo hàm phức tạp hơn

↔ **trường hợp riêng của C** — mọi bài ước lượng tư thế bằng mạng là một trường hợp riêng của việc chọn biểu diễn liên tục cho đầu ra trên đa tạp

Cách sáu số cho mạng xuất ra hai vector ba chiều rồi trực giao hoá. Nó được phát biểu rõ trong dòng công trình về “biểu diễn quay liên tục” của Zhou và cộng sự. Cách chín số rồi chiếu bằng SVD tương đương về tinh thần, và dùng đúng kết quả xấp xỉ hạng thấp ở [A/01/01](#).

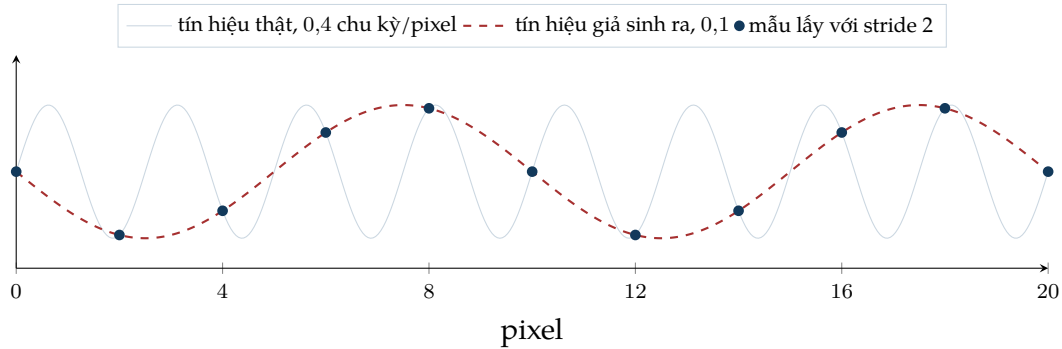
Một chi tiết dễ nhầm và tốn thời gian debug: *lấy trung bình các phép quay*. Trung bình cộng của các ma trận quay không phải ma trận quay; trung bình quaternion phụ thuộc dấu bạn chọn; trung bình góc Euler vô nghĩa qua chỗ chuyển $\pm\pi$. Cách đúng là trung bình trên không gian tiếp tuyến quanh một điểm neo rồi ánh xạ về, hoặc chiếu trung bình ma trận về $SO(3)$ bằng SVD. Sai lầm này xuất hiện lại nguyên vẹn khi người ta gộp dự đoán của nhiều mô hình hay nhiều *test-time augmentation*.

C. Tín hiệu: mọi lỗi im lặng đều là lỗi lấy mẫu

Convolution là phép toán duy nhất vừa tuyến tính vừa **đẳng biến tịnh tiến** (*translation equivariance*): dịch ảnh rồi lọc cũng bằng lọc rồi dịch. Đó là toàn bộ lý do CNN tồn tại. Ta *cài sẵn* giả định “một cạnh vẫn là một cạnh khi dịch sang phải mười pixel”, thay vì bắt mạng học nó từ dữ liệu. Định lý convolution nói convolution trong miền không gian là phép nhân từng điểm trong miền Fourier; nói bằng lời, *mỗi kernel là một bộ lọc tần số* — kernel làm mịn là thông thấp, kernel Sobel là thông cao. Từ đó cả kiến trúc đọc được gọn: một CNN sâu là chuỗi lọc xen kẽ các bước giảm mẫu, và mỗi bước giảm mẫu vớt bỏ một nửa dải tần.

Chính chỗ vớt bỏ ấy là nơi mọi thứ hỏng. **Định lý lấy mẫu** (*sampling theorem*) nói tần số lấy mẫu phải lớn hơn hai lần tần số cao nhất có trong tín hiệu; nếu không, thành phần tần số cao không biến mất mà *gập* xuống thành tần số thấp giả.

Ví dụ tính tay. Hoa văn 0,4 chu kỳ trên pixel, lấy mẫu cách hai pixel (stride 2): tần số lấy mẫu 0,5, tần số Nyquist 0,25; thành phần 0,4 vượt ngưỡng nên gập về $|0,4 - 0,5| = 0,1$ chu kỳ trên pixel — một hoa văn thô gấp bốn lần, không hề tồn tại trên vật thật. Hình 1.6 vẽ đúng phép tính đó.



Hình 1.6. Aliasing không làm mất thông tin một cách trung thực, nó *bịa* thông tin. Các điểm lấy mẫu cách hai pixel nằm chính xác trên một hình sin tần số thấp hơn bốn lần; mọi tầng phía sau đều không có cách nào biết hoa văn thô ấy là giả.

→ **tiền đề** A/02 — khử khảm và giảm mẫu màu 4:2:0 trong chuỗi ISP là đúng bài toán lấy mẫu này, chỉ đổi tên

Từ đó rút ra quy tắc bất di bất dịch của mọi phép giảm mẫu: **lọc thông thấp trước, rồi mới lấy mẫu thưa**. Hệ quả ở hai chỗ:

- **Trong dataloader** — thu nhỏ ảnh phải dùng nội suy vùng (INTER_AREA) hoặc làm mờ Gaussian trước; *nearest-neighbor* gần như luôn sai khi thu nhỏ. Đây là một dòng code, và nó đáng vài điểm độ chính xác.
- **Trong kiến trúc** — tầng stride-2 và *max-pooling* lấy mẫu thưa mà không lọc trước, nên CNN *không* bất biến dịch chuyển như người ta hay nói: dịch ảnh một pixel có thể đổi hẳn dự đoán. Dòng công trình “làm cho CNN bất biến dịch trở lại” của Zhang chèn một bộ lọc mờ ngay trước mỗi bước giảm mẫu. Cách đó khôi phục phần lớn tính bất biến. Kết quả đã được đo và tái lập, dù mức lợi ích tùy bài toán.

Góc nhìn Fourier còn giải thích hai hiện tượng nữa. Thứ nhất là *spectral bias*. Mạng khớp thành phần tần số thấp trước, tần số cao sau, nên chi tiết mảnh xuất hiện muộn trong huấn luyện. Khi mục tiêu vốn nhiều tần số cao — chẳng hạn hàm mật độ của một cảnh 3D trong *NeRF* — người ta phải mã hoá đầu vào bằng đặc trưng Fourier để mạng học nổi [28].

→ **tiền đề** C — đẳng biến tịnh tiến là thiên lệch quy nạp mà cả dòng CNN dựa vào, và việc nó không đúng hoàn toàn là một trong các động cơ dẫn tới *attention*

Thứ hai là chuyện thang đo: một mô hình huấn luyện ở 224×224 thấy các cạnh ở một dải tần nhất định; đưa vào ảnh 4K rồi crop, cùng vật thể xuất hiện ở dải tần khác hẳn, và mô hình sứt điểm dù “vẫn là ảnh của cùng cái xe”.

1 MẠCH 3 — TIỀN XỬ LÝ ẢNH, VÀ CÁI GIÁ CỦA MỖI BƯỚC

- 1 Vấn đề** ảnh vào có kích thước bất kỳ, mạng cần kích thước cố định.

Giải pháp resize.

Giả định nội suy giữ nguyên nội dung ngữ nghĩa.

Giá phải trả đổi phổ tần số của ảnh; thu nhỏ mà không lọc trước thì sinh hoa văn giả.
- 2 Vấn đề** resize làm méo tỉ lệ khung hình.

Giải pháp *letterbox* — giữ tỉ lệ, đệm phần thiếu.

Giá phải trả đưa vào vùng đệm nhân tạo mà mạng có thể học như tín hiệu, và mọi tọa độ hộp phải được biến đổi ngược đúng cách — một chỗ sinh bug rất phổ biến.
- 3 Vấn đề** muốn dữ liệu phong phú hơn nên xoay, lật, cắt.

Giả định	phép biến đổi không đổi nhãn.
Vấn đề mới	giả định đó sai cho biểu báo có chữ, cho ảnh y tế có tính trái-phải, và cho mọi bài toán phải dự đoán hướng; ngoài ra mỗi lần xoay là một lần nội suy, tức lại đổi phổ.
4 Vấn đề	muốn augmentation mạnh hơn nên chồng nhiều phép liên tiếp.
Giả định	các phép nội suy chồng nhau không tích lũy hại.
Giá phải trả	sai — mỗi lần nội suy làm mờ thêm.
Giải pháp	gộp các phép biến đổi thành một ma trận rồi lấy mẫu <i>một lần</i> duy nhất, đúng như cách bạn gộp chuỗi $SE(3)$ thành một phép biến đổi trước khi warp.

D. Giới hạn và trường hợp hỏng

- **Bức tranh Fourier giả định tuyến tính và bất biến dịch**, mà mạng thật có phi tuyến sau mỗi tầng. “Mỗi tầng là một bộ lọc tần số” là trực giác hướng dẫn tốt, không phải mô tả chính xác — đừng suy diễn định lượng từ nó.
- **Hình học ở trên giả định phép biến đổi cứng và camera lý tưởng**. *Rolling shutter* phá giả định “một ảnh ứng với một pose” vì mỗi hàng pixel chụp ở một thời điểm khác; ống kính có méo làm phép biến đổi giữa hai ảnh không còn là homography ngay cả với mặt phẳng; vật thể không cứng thì $SE(3)$ không mô tả nổi. Mỗi giả định vỡ ở một chỗ và đòi một cách vá khác nhau — chủ đề của A/02/03.

CẦN NHỚ

Hai câu hỏi phòng vệ, hỏi mỗi lần: *trước khi giảm độ phân giải — đã lọc thông thấp chưa?* và *trước khi cho mạng xuất ra phép quay — biểu diễn này có liên tục không?* Chúng rõ như một dòng code và bắt được phần lớn lỗi im lặng ở khâu hình học và tín hiệu. [T1]

E. Kết nối

Mục này là nền trực tiếp của cả chương A/02-vat-ly-tao-anh: phần lấy mẫu giải thích vì sao **khử khảm** (*demosaiçing*) là một bài toán khôi phục chứ không phải nội suy tầm thường, và phần hình học là toàn bộ ngôn ngữ của A/02/03. Theo chiều ngang, tính bất biến dịch chuyển là giả định thiết kế của dòng CNN ở C, và việc nó *không* đúng hoàn toàn là một trong các động cơ dẫn tới *attention*; còn phần biểu diễn quay quay lại ở mọi bài toán ước lượng tư thế bằng mạng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao không tồn tại cách cho mạng xuất ra phép quay bằng bốn số mà không có điểm gián đoạn, và cách sáu số sửa được gì?
2. Hoa văn 0,45 chu kỳ/pixel bị lấy mẫu với stride 2 sẽ hiện ra ở tần số nào? Tính tay.
3. Vì sao CNN không thật sự bất biến dịch chuyển, dù convolution thì có? Chỗ phá vỡ nằm ở tầng nào?
4. Bạn chồng năm phép augmentation hình học liên tiếp, mỗi phép một lần nội suy. Hại gì xảy ra và sửa thế nào?
5. Vì sao lấy trung bình cộng các ma trận quay là sai, và cách đúng dựa trên kết quả nào ở mục đại số tuyến tính?
6. Một mô hình huấn luyện ở 224×224 sụ điểm khi chạy trên ảnh 4K đã crop về 224×224 — giải thích bằng ngôn ngữ tần số thì chuyển gì đã xảy ra?

Nguồn

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.

1.5 Thực hành

Phần lý thuyết ở trên chỉ trở thành kiến thức dùng được sau khi bạn tự tay chạm vào một lần. Khối dưới đây được thiết kế để tốn ít thời gian nhất mà chạm được vào nhiều điểm nhất; hãy làm mini project ngay cả khi thấy đã hiểu, vì câu hỏi kết thúc nó là câu hỏi mà lý thuyết một mình không trả lời được.

Repo và tài nguyên. Cho phần lý thuyết, *Mathematics for Machine Learning* [2] có PDF miễn phí và đúng liều lượng cho mục tiêu ở đây. Khi cần đổi chiều một đạo hàm ma trận, matrixcalculus.org trả lời nhanh hơn tra sách; cần chắc chắn hơn nữa thì [sympy](https://sympy.org/) cho phép derive tay có kiểm chứng ký hiệu. Đạo hàm tự viết phải kiểm bằng `torch.autograd.gradcheck` trên đầu vào kiểu double — kiểm bằng float32 hay báo sai giả. Ghi chú lan truyền ngược của CS231n vẫn là bản giải thích đồ thị tính toán tốt nhất ở mức nhập môn, dù danh mục kiến trúc trong cùng khoá học thì nên đọc như lịch sử tư duy. Nhóm tài nguyên này ít bị bán rã — sách toán và công cụ ký hiệu bền hơn nhiều so với tên mô hình dẫn đầu bảng, vốn sẽ cũ trong 6–12 tháng kể từ mốc chốt tháng 9/2026.

Câu hỏi kiểm tra hiểu. Ba câu, mỗi câu chạm một mục khác nhau:

- Vì sao KL divergence không đối xứng, và sự bất đối xứng đó khiến một mô hình sinh trái mỏng hay chọn một mode?
- Số điều kiện lớn của một ma trận trọng số nói gì về độ ổn định huấn luyện — và nó *không* nói gì?
- Với tập test 500 mẫu và độ chính xác 90%, khoảng tin cậy 95% rộng bao nhiêu, và chênh lệch một điểm phần trăm giữa hai mô hình có nghĩa không? Câu trả lời khác nhau ra sao giữa hai phép đo độc lập và một phép so sánh ghép cặp trên cùng tập test?

Bài tập phụ. Cài PCA và *whitening* bằng NumPy thuần trên CIFAR-10 — tự tính ma trận hiệp phương sai, tự phân rã, hiển thị 16 vector riêng đầu tiên dưới dạng ảnh — rồi trả lời: vì sao whitening gần như không được dùng trong pipeline thị giác hiện đại? Ba lý do đáng tự tìm trước khi đọc gợi ý: chuẩn hoá theo tầng bên trong mạng đã làm một việc tương đương nhưng thích nghi hơn; whitening khuếch đại đúng những hướng phương sai nhỏ vốn chủ yếu là nhiễu cảm biến; và thống kê whitening tính trên tập huấn luyện trở thành một cái bẫy dịch chuyển miền khi triển khai trên camera khác. Nửa ngày, và nó xa SLAM — bài chính của chương nằm ở khối mini project bên dưới.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Phép chiếu vuông góc	Phép hạ một vector xuống không gian con gần nó nhất, sao cho phần dư vuông góc với không gian đó. Mọi bài bình phương tối thiểu là một phép chiếu; phương trình chuẩn tắc chỉ là cách viết lại điều kiện vuông góc.
•• SVD	Phân rã giá trị kỳ dị: cách viết một ma trận bất kỳ thành xoay, kéo dãn theo vài trục, rồi xoay lần nữa. Dùng để đọc hạng, để xấp xỉ hạng thấp, và để ép một ma trận nhiều về $SO(3)$.
••• Số điều kiện	Tỉ số giữa trị kỳ dị lớn nhất và nhỏ nhất. Nó là hệ số khuếch đại sai số của <i>bài toán</i> , nên đối solver hay đối optimizer không cứu được. Nó vô nghĩa khi so giữa hai cách chọn đơn vị đo khác nhau.
•• Xấp xỉ hạng thấp	Giữ lại k trị kỳ dị lớn nhất để có ma trận hạng k gần bản gốc nhất. Đứng sau PCA, sau phép chiếu về $SO(3)$, và sau LoRA.
• Hạng hiệu dụng	Entropy của phổ trị kỳ dị đã chuẩn hoá. Cần nó vì với số đầu thấy động hầu như không trị kỳ dị nào bằng đúng không, nên đếm số trị khác không là vô nghĩa.
•• Vì sao lập $A^T A$ làm mất chữ số có nghĩa?	Vì trị kỳ dị bị bình phương, nên số điều kiện cũng bình phương. Bạn mất khoảng một nửa số chữ số; QR hoặc SVD giải thẳng trên A thì không.
••• Lan truyền ngược	Thuật toán tính gradient bằng cách đi ngược đồ thị phép tính đúng một lượt. Nó rẻ vì mỗi lượt trả về đạo hàm theo <i>mọi</i> tham số, nhưng phải giữ lại giá trị trung gian của lượt xuôi nên tốn bộ nhớ.
••• Lan truyền ngược so với lượt ngược	Lan truyền ngược là <i>thuật toán</i> ; lượt ngược là <i>một lần chạy</i> thuật toán đó trên một batch. Hai từ này hay bị dùng lẫn.
••• Quy tắc khớp shape	Gradient của một đại lượng phải có đúng shape của chính nó, nên thường chỉ còn một cách ghép hợp lệ. Khớp shape trước, đầu và hệ số sau; phần còn lại kiểm bằng gradcheck ở kiểu double.
•• Điểm yên ngựa	Điểm có gradient bằng không nhưng độ cong âm theo ít nhất một hướng. Trong không gian nhiều triệu chiều, gần như mọi điểm dừng đều là yên ngựa chứ không phải cực tiểu địa phương.
•• Vì sao chuẩn hoá và Adam làm huấn luyện nhanh hơn?	Cả hai kéo các hướng về cùng thang đo, tức hạ số điều kiện một cách gián tiếp. Khi các trục gần bằng nhau, một learning rate chung mới vừa cho mọi hướng.
•• Loss thành NaN sau vài trăm bước — nguyên nhân?	Learning rate quá lớn, hoặc tràn số ở độ chính xác 16 bit. Phân biệt: hạ learning rate mười lần mà vẫn NaN thì là tràn số, hãy bật loss scaling hoặc đổi sang bf16.
••• Độ hợp lý	Xác suất mà mô hình gán cho dữ liệu đã quan sát, xem như hàm của tham số. Cực đại nó tương đương cực tiểu âm log của nó — và đó chính là gần hết các hàm mất mát đang dùng.
••• Sai số chuẩn	Với độ chính xác đo trên n mẫu, nó bằng $\sqrt{p(1-p)/n}$. Với $n = 500$ và $p = 0.9$ thì bằng 1,34 điểm, nên khoảng tin cậy 95% rộng hơn năm điểm. Nó phình to khi các mẫu không độc lập.
•• Phân kỳ KL	Chi phí phụ trội khi mã hoá dữ liệu của phân phối này bằng bảng mã của phân phối kia. Không đối xứng: chiều xuôi ép mô hình phủ mọi mode, chiều ngược ép nó bám lấy một mode.
•• Vì sao hồi quy độ sâu bằng MSE cho ra biên mờ?	Vì ở biên, độ sâu thật là đa phương thức, còn MSE giả định Gaussian đơn phương thức. Mô hình buộc phải trả về trung bình của hai đáp án đúng, tức một giá trị sai ở giữa.
• Loss ban đầu của bài 1000 lớp bằng 2 — nghĩa là gì?	Nghĩ có rò rỉ nhãn. Khởi tạo hợp lý phải cho khoảng $\ln 1000 \approx 6.9$; nếu ngược lại nó bằng 20 thì lỗi nằm ở khởi tạo hoặc ở tầng cuối.
••• Aliasing	Hiện tượng thành phần tần số cao <i>gấp</i> xuống thành tần số thấp giả khi lấy mẫu quá thưa. Nó không làm mất thông tin một cách trung thực mà bịa thêm cấu trúc không có thật.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Đẳng biến tịnh tiến	Tính chất dịch ảnh rồi lọc cũng bằng lọc rồi dịch. Convolution có tính chất này, nhưng bước stride và pooling phá nó, nên CNN không thật sự bất biến với dịch chuyển.
•• Vì sao không cho mạng xuất ra quaternion?	Vì q và $-q$ là cùng một phép quay, nên hàm mục tiêu gián đoạn, mà một mạng liên tục thì không khớp nổi hàm gián đoạn. Cách sáu số rồi trực giao hoá thì liên tục.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
PyTorch, JAX	Toàn bộ <i>autograd</i> là hiện thực của vi phân chế độ ngược trên đồ thị tính toán — đúng mạch ở mục 2	Hạ tầng, ổn định từ 2017; ít bán rã nhất trong bảng này
Ceres, g2o, GTSAM	Bình phương tối thiểu phi tuyến với phân rã thừa và damping — phép chiếu cộng điều hoà số học của mục 1	Hạ tầng SLAM; chính là chỗ nổi với nền sẵn có của bạn
LoRA và họ tinh chỉnh hiệu quả tham số	Giải thiết ma trận cập nhật trọng số có hạng thấp, tức định lý xấp xỉ hạng thấp	Cách tinh chỉnh mặc định cho mô hình lớn tính tới 9/2026
AdamW	Tách weight decay khỏi bước thích nghi — hệ quả trực tiếp của việc Adam đổi thang đo từng toạ độ	Optimizer mặc định của phần lớn mô hình lớn hiện nay
Đặc trưng Fourier trong NeRF và họ biểu diễn ngầm	Mã hoá đầu vào bằng nhiều tần số, vì mạng vốn khớp tần số thấp trước	Ý tưởng còn sống trong các biến thể dựng cảnh gần đây

MINI PROJECT SLAM · Đo số điều kiện của bài toán tư thế trên hai chuỗi EuRoC · vài giờ tới một ngày

Mục tiêu	biến ba khái niệm trừu tượng nhất của chương — phép chiếu, SVD, số điều kiện — thành ba con số đọc được trên chính hệ SLAM bạn đang chạy. Câu cần trả lời: chuỗi nào khó hơn, và khó vì <i>hình học chuyển động</i> chứ không vì thuật toán.
Dữ liệu	hai chuỗi EuRoC có đặc tính chuyển động khác hẳn nhau — một chuỗi tịnh tiến rộng và một chuỗi có nhiều đoạn xoay gần tại chỗ — cùng chân trị tư thế đi kèm tập dữ liệu. Không cần dữ liệu tự quay.
Các bước	(1) với mỗi <i>keyframe</i> , lấy chính ma trận Jacobian mà solver của bạn đã dựng cho bài tối ưu tư thế, hoặc dựng lại bằng cách xếp các hàng ràng buộc sai số tái chiếu của những điểm đang bám được. (2) Cân lại thang đo hai khối biến trước khi so sánh — tịnh tiến tính bằng mét, quay tính bằng radian — vì nếu không thì số điều kiện chỉ phản ánh lựa chọn đơn vị. (3) Tính phổ trị kỳ dị bằng SVD và ghi $\kappa = \sigma_{\max}/\sigma_{\min}$ theo thời gian cho cả hai chuỗi. (4) Ở mỗi chỗ phải chuẩn hoá một ma trận quay ước lượng, hãy làm bằng SVD: đặt cả ba trị kỳ dị bằng một. Ghi lại độ lệch trực giao <i>trước</i> khi chiếu, như một chỉ báo sức khoẻ số học. (5) Tính <i>RPE</i> bằng <i>evo</i> trên cửa sổ trượt để có một đường sai số cùng trục thời gian.
Xong khi	bạn có một hình duy nhất, vẽ hai đường κ theo thời gian trên cùng trục. Trên đó bạn chỉ ra các đoạn κ vọt lên, và đối chiếu chúng với đoạn chuyển động gần thuần quay hoặc điểm gần đồng phẳng. Cuối cùng bạn nêu một ngưỡng κ mà vượt qua thì RPE cửa sổ trượt tăng rõ trên cả hai chuỗi.
Nổi với	A/01/01 cho SVD và số điều kiện; A/01/04 cho phép chiếu về $SO(3)$; A/02/03 cho lý do hình học khiến cấu hình gần thuần quay là suy biến. Đường κ theo thời gian sinh ra ở đây được dùng lại làm biến giải thích trong mini project của chương 2.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì chi phí nằm ở *thứ tự* nhân các Jacobian, không ở quy tắc chuỗi. Nhân ngược từ phía đầu ra vô hướng cho mọi đạo hàm trong một lượt; nhân theo chiều xuôi thì tốn một lượt cho mỗi tham số.
- A2. Vì số điều kiện đo tỉ số giữa trục dài nhất và trục ngắn nhất của bài toán *sau khi đã chọn đơn vị*; đổi đơn vị chỉ co giãn các trục chứ không đổi lượng thông tin mà dữ liệu ràng buộc. Nên chỉ so sánh được khi cách tham số hoá giữ nguyên.
- A3. Vì đặt hai khoảng tin cậy độc lập cạnh nhau là phép so sánh sai. Trên cùng một tập test, hai mô hình sai ở phần lớn cùng những mẫu khó, nên chỉ các mẫu *bất đồng* mới mang thông tin — và kiểm định ghép cặp trên chúng có độ bất định nhỏ hơn nhiều.
- A4. Vì không tồn tại phép tham số hoá liên tục của $SO(3)$ bằng ba hay bốn số: quaternion phủ hai lần nên hàm mục tiêu gián đoạn, mà một mạng liên tục theo đầu vào thì không khớp nổi hàm gián đoạn. Sáu số rồi trục giao hoá thì liên tục.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn phải tìm chỗ thấp nhất của một thung lũng, giữa sương mù dày, và chỉ nhìn được mặt đất ngay dưới chân. Cách duy nhất là sờ xem phía nào dốc xuống rồi bước một bước về phía đó. Chương này là ba câu hỏi nảy ra từ đúng tình huống ấy.

Thung lũng này hình gì? Nhiều thung lũng thật ra là một khe rất hẹp và rất dài: đi ngang thì dốc đứng, đi dọc thì gần như bằng phẳng. Không biết điều đó thì bạn cứ nảy qua nảy lại giữa hai vách mà chẳng tiến xuống bao nhiêu. Có một phép tính cho biết khe hẹp gấp bao nhiêu lần chiều dài của nó, và con số ấy dự báo trước bạn sẽ phải chờ lâu cỡ nào.

Bước bao nhiêu là vừa? Bước dài thì văng qua vách bên kia, bước ngắn thì cả đời không tới đáy.

Làm sao biết mình đã xuống thật? Cái đồng hồ đo độ cao trong tay có sai số. Nếu nó sai chừng hai mét mà bạn khoe vừa xuống thêm một mét, thì bạn chưa khoe được gì.

Cái giá: mọi mẹo trong chương chỉ làm khe rộng ra một chút cho dễ đi, chứ không ai vén được sương mù.

Chỗ không rút gọn được: vì sao dừng ở một cái đáy rộng lại cho kết quả bền hơn dừng ở đáy nhọn thì tới nay chưa có lời giải thích vừa gọn vừa đúng — và ngay cả chữ “rộng” cũng đổi nghĩa khi ta đổi cách viết các con số mô tả mô hình.

Một câu: Toán ở chương này chỉ để trả lời ba câu: thung lũng hẹp theo hướng nào, bước bao nhiêu thì an toàn, và khi nào cái đồng hồ đo mới thật sự nói rằng bạn đã xuống thấp hơn.

1.6 Kết nối

Chương này là tiền đề kỹ thuật của cả cây, nhưng nó nổi ra ngoài theo ba hướng rất khác nhau. Hướng thứ nhất đi sang A/02-vat-ly-tao-anh: phần lấy mẫu và phần hình học của mục 4 được dùng gần như liên tục ở đó, nên đây là nhánh nên đọc tiếp ngay. Hướng thứ hai đi sang Phần B, nơi các đại lượng ở đây đổi vai. Số điều kiện và bộ nhớ kích hoạt không còn là tính chất toán học; chúng trở thành *ràng buộc thiết kế*, đứng cạnh ràng buộc về nhân và về độ trễ. Hướng thứ ba đi sang E: toàn bộ mục 3 là bộ công cụ mà chương đánh giá dùng hằng ngày, và nếu bỏ qua nó thì mọi bảng kết quả về sau đều đọc sai.

Tổng kết chương

Chương này đã dựng bốn công cụ và, quan trọng hơn, đã gắn mỗi công cụ vào một câu hỏi cụ thể sẽ quay lại. Không gian con và phép chiếu cho ta cách đọc mọi bài bình phương tối thiểu; phổ và số điều kiện cho ta ngôn ngữ chung giữa “bài toán suy biến” trong ước lượng trạng thái và “landscape khó tối ưu” trong huấn luyện. Bộ máy vi phân cho ta quy tắc khớp shape trước rồi mới lo dấu, cùng hiểu biết vì sao lan truyền ngược đặt về bộ nhớ chứ không về phép tính. Xác

suất và lý thuyết thông tin cho thấy hàm mất mát không phải phát minh rời rạc. Nó là âm log độ hợp lý của một giả định phân phối. Kèm theo đó là thói quen tính sai số trước khi tin một con số. Hình học và tín hiệu cho ta cấu trúc có sẵn của dữ liệu ảnh cùng hai câu hỏi phòng vệ: đã lọc trước khi giảm mẫu chưa, và biểu diễn quay này có liên tục không.

Còn bỏ ngỏ khá nhiều, và bỏ ngỏ có chủ ý. Chương này không đụng tới lý thuyết học thống kê — vì sao học được từ mẫu hữu hạn — đó là nội dung của Phần B. Nó không đụng tới tối ưu hoá lỗi ở mức chứng minh, không đụng tới giải tích ngẫu nhiên hay lý thuyết ma trận ngẫu nhiên, và cố ý chỉ chạm Lie group ở đúng ba thao tác dùng được. Nếu sau này bạn thấy cần sâu hơn ở một nhánh, đó là dấu hiệu tốt: nhu cầu thật xuất hiện trước, việc học đến sau. Nhánh nên đọc tiếp ngay là A/02-vat-1y-tao-anh, nơi phần lấy mẫu và phần hình học của mục 4 được dùng liên tục.

! Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Mục 1 nói số điều kiện là tính chất của *bài toán* nên đổi solver không cứu được, còn mục 2 nói chuẩn hoá và Adam làm huấn luyện nhanh hơn nhờ hạ số điều kiện. Hai phát biểu này mâu thuẫn ở chỗ nào, và giả định nào khác nhau khiến cả hai cùng đúng?
2. Bài bình phương tối thiểu của mục 1 và âm log độ hợp lý Gauss của mục 3 là cùng một thứ viết hai cách. Nếu đổi giả định nhiều sang một phân phối đuôi nặng, công cụ nào của mục 1 — phương trình chuẩn tắc, phép chiếu, số điều kiện — còn dùng được và cái nào mất?
3. Bản đồ độ sâu của bạn mờ dọc theo mọi biên vật thể. Mục 3 và mục 4 cho hai nguyên nhân khác hẳn nhau cho cùng triệu chứng đó; hai nguyên nhân là gì, và phép thử nào phân biệt chúng mà không cần huấn luyện lại?
4. Bạn cho mạng xuất sáu số rồi trực giao hoá bằng SVD để có một phép quay. Thao tác đó nằm trong đồ thị tính toán, nên mục 2 đòi hỏi thêm điều gì mà mục 4 chưa nói tới, và chuyện gì xảy ra khi hai trị kỳ dị gần bằng nhau?
5. Ghép điều kiện dừng của mục 2 với sai số chuẩn của mục 3: vì sao “val loss không giảm thêm sau nhiều epoch” là quy tắc dừng có nghĩa hơn “chuẩn gradient nhỏ”, và quy tắc đó vẫn hỏng trong trường hợp nào?
6. Whitening bằng PCA và chuẩn hoá bên trong mạng đều nhằm kéo các hướng về cùng thang đo. Vì sao chỉ cái sau sống sót trong pipeline hiện đại, và lý do đó liên quan thế nào tới việc phổ trị riêng thay đổi trong lúc huấn luyện?
7. Đường κ theo thời gian trong mini project chỉ đọc được sau khi bạn cân lại thang đo giữa khối tịnh tiến và khối quay. Nếu bỏ bước cân đó, đường κ có còn vọt lên ở các đoạn gần thuần quay không, và khi đó bạn đang đo cái gì?

Nguồn

- [1] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997.
- [2] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [3] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [4] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015.
- [5] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.
- [6] L. van der Maaten and G. Hinton, “Visualizing Data Using t-SNE,” *JMLR*, vol. 9, pp. 2579–2605, 2008.
- [7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv:1802.03426*, 2018.
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [9] T. Chen, B. Xu, C. Zhang, et al., “Training Deep Nets with Sublinear Memory Cost,” *arXiv:1604.06174*, 2016.
- [10] S. L. Smith, P.-J. Kindermans, C. Ying, et al., “Don’t Decay the Learning Rate, Increase the Batch Size,” in *ICLR*, 2018.
- [11] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019.
- [12] N. S. Keskar, D. Mudigere, J. Nocedal, et al., “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in *ICLR*, 2017.
- [13] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.
- [14] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [15] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [16] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [17] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” *JMLR*, vol. 7, pp. 1–30, 2006.
- [18] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [19] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [20] T.-Y. Lin, P. Goyal, R. Girshick, et al., “Focal Loss for Dense Object Detection,” in *ICCV*, 2017.
- [21] C. Szegedy, V. Vanhoucke, S. Ioffe, et al., “Rethinking the Inception Architecture for Computer Vision,” in *CVPR*, 2016.
- [22] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *ICLR*, 2014.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.

- [24] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [25] A. N. Angelopoulos and S. Bates, *Conformal Prediction: A Gentle Introduction*. Foundations and Trends in Machine Learning, 2023.
- [26] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [27] J. Solà, J. Deray, and D. Atchuthan, “A micro Lie theory for state estimation in robotics,” *arXiv:1812.01537*, 2018.
- [28] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.